INTRODUCTION TO PROGRAMMING • DIGITAL NOTES

UNIT I

Introduction to Programming & Problem Solving

Computers • Languages • Algorithms • Flowcharts • Data Types • I/O • Operators • Complexity

▶ What you will learn in this unit

- History & generations of computers
- Basic organization: ALU, CU, memory, PC, I/O
- Programming languages
- Basics of a computer program
- Algorithms, flowcharts (Dia) & pseudocode
- Compilation & execution
- Data types, variables & constants
- Basic input / output
- Operators, type conversion & casting
- Problem solving: top-down / bottom-up
- Characteristics of an algorithm
- Time & space complexity

1.1 History of Computers

◆ Definition

A **computer** is an electronic machine that accepts **data** (input), processes it using stored **instructions** (program), and produces **information** (output).

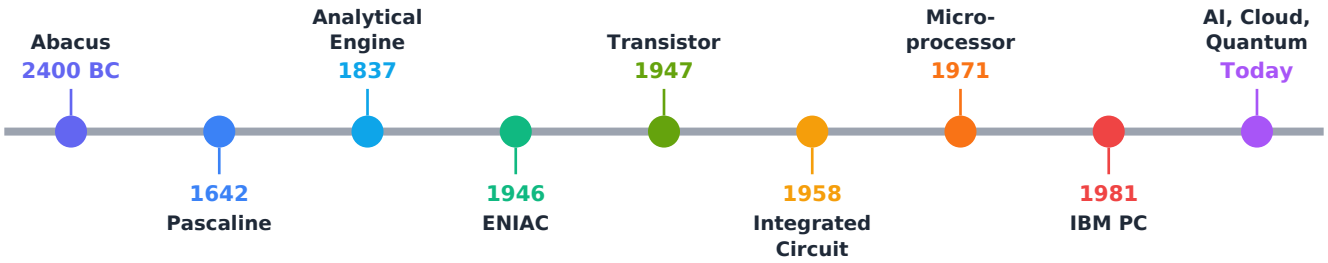


Fig 1.1 — Milestones in the history of computing

Gen.	Period	Technology	Features / Examples
1st	1940–56	Vacuum tubes	Huge, hot, slow, machine language — ENIAC, UNIVAC
2nd	1956–63	Transistors	Smaller, faster, assembly language — IBM 1401
3rd	1964–71	Integrated Circuits (IC)	High-level languages, operating systems — IBM 360
4th	1971–today	Microprocessor (VLSI)	Personal computers, GUI, networks — IBM PC, laptops
5th	Present & beyond	ULSI, AI, Quantum	Artificial intelligence, voice recognition, robotics

✔ Remember

Easy way to remember: Tubes → Transistors → ICs → Microprocessors → AI. Each generation made computers **smaller, faster, cheaper and more reliable**. *Charles Babbage* is the "Father of the Computer"; *Ada Lovelace* wrote the first program.

Characteristic	Meaning	Application area	Use
Speed	Billions of operations per second	Education	e-learning, online exams
Accuracy	Errors only from wrong input or program	Banking	ATMs, online payment
Storage	Stores huge data permanently	Healthcare	Scans, patient records
Automation	Runs without human help	Science	Simulation, satellites

1.2 Basic Organization of a Computer

Every computer, from a phone to a supercomputer, follows the **Von Neumann architecture**: programs and data are stored together in memory, and the CPU fetches and executes them one by one.

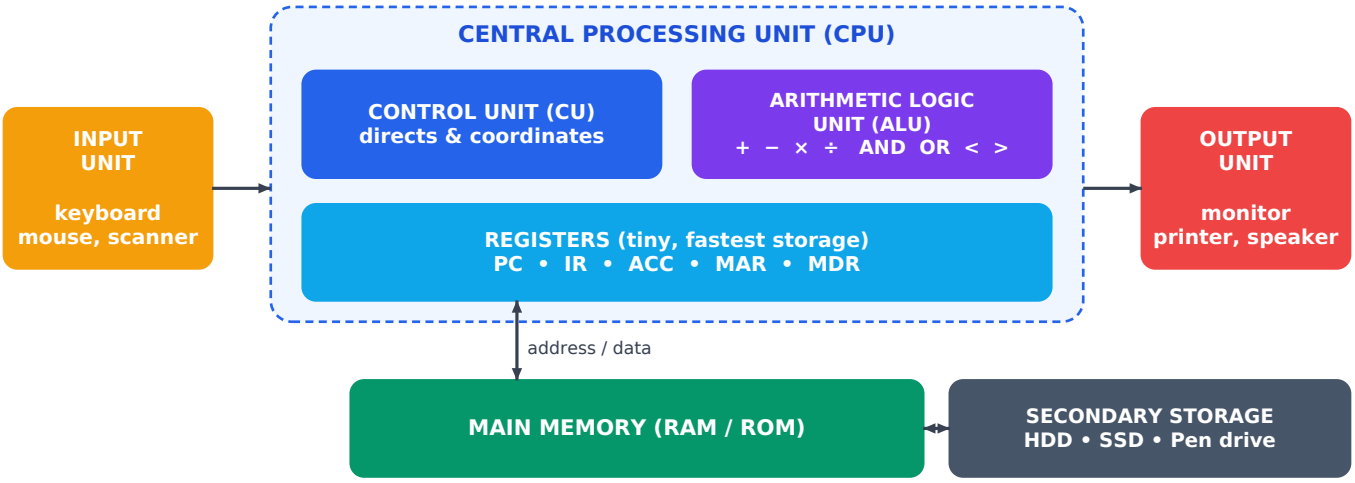


Fig 1.2 — Block diagram of a computer system

Unit	Function	Example
Input unit	Takes data/instructions from the user and converts them into binary	Keyboard, mouse, scanner, mic
ALU	Performs **arithmetic** (+ − × ÷) and **logical** (AND, OR, NOT, comparisons) operations	Adds two numbers, checks a > b
Control Unit	Fetches instructions, decodes them, and sends control signals to all other parts	The "manager" of the CPU
Memory	Holds programs and data. **Primary** (RAM, ROM, cache) is fast; **secondary** is permanent and large	RAM, HDD, SSD
Output unit	Converts binary results into human-readable form	Monitor, printer, speaker

Unit	Size	Unit	Size	Memory hierarchy (fast & small → slow & large)
1 bit	0 or 1	1 MB	1024 KB	Registers → Cache → RAM → SSD/HDD
1 byte	8 bits	1 GB	1024 MB	Faster = costlier = smaller capacity
1 KB	1024 bytes	1 TB	1024 GB	RAM is **volatile**; ROM / disk are **non-volatile**

Program Counter (PC) and Important Registers

- **Program Counter (PC)** — holds the **address of the next instruction** to be executed. After every fetch it is automatically increased by 1 (instruction length).
- **Instruction Register (IR)** — holds the instruction currently being decoded.
- **Accumulator (ACC)** — stores intermediate results of the ALU.
- **MAR / MDR** — Memory Address Register / Memory Data Register: where to read/write, and what to read/write.

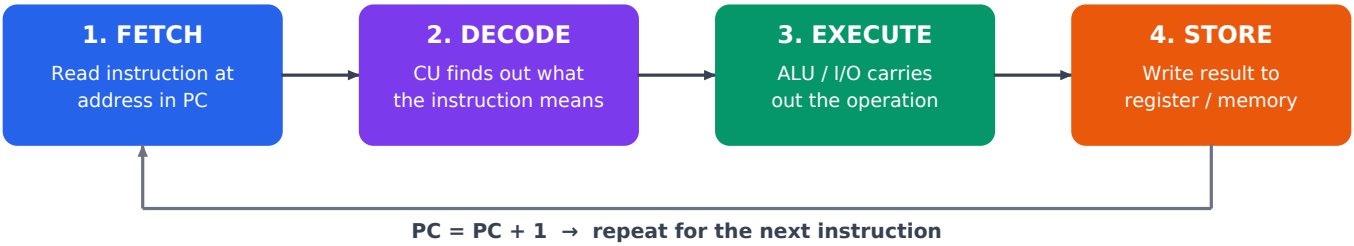


Fig 1.3 — Instruction cycle (Fetch–Decode–Execute)

1.3 Introduction to Programming Languages

A **programming language** is a set of words and rules used to give instructions to a computer. A **program** is a finite set of such instructions that solves a problem.

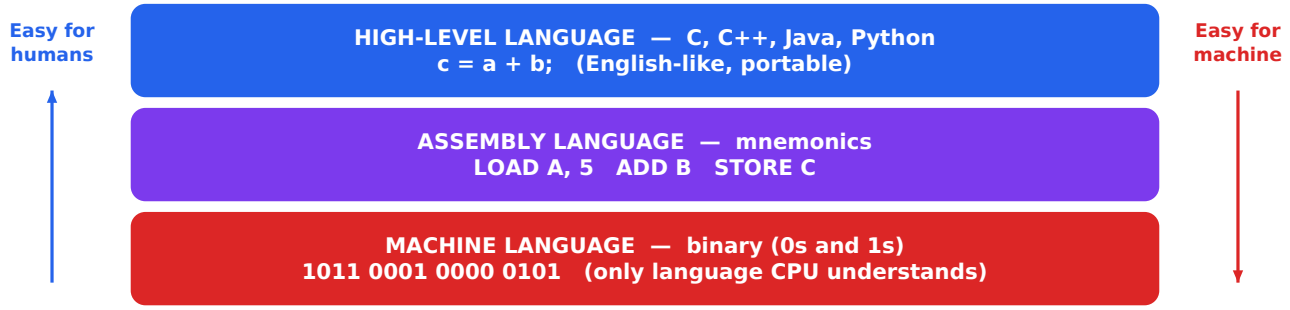


Fig 1.4 — Levels of programming languages (translators convert top → bottom)

	Machine	Assembly	High-level
Form	0s and 1s	Mnemonics (ADD, MOV)	English-like statements
Translator	None needed	Assembler	Compiler / Interpreter
Speed of run	Fastest	Very fast	Slightly slower
Portability	Machine dependent	Machine dependent	Machine independent
Ease	Very hard	Hard	Easy

Translator	How it works	Example
Compiler	Translates the **whole program** at once into machine code; reports all errors together	C, C++ (gcc)
Interpreter	Translates and runs **line by line**; stops at the first error	Python, JavaScript
Assembler	Converts assembly language to machine code	NASM, MASM

● Real-life Use
C is used to build operating systems (Linux, Windows kernel), embedded systems (washing machines, cars), compilers, databases and device drivers because it is **fast and close to hardware**.

1.4 Basics of a Computer Program

A C program is a collection of **functions**. Execution always starts from `main()`. Below is the smallest useful program with every part labelled.

sum.c

```c
#include <stdio.h>      /* 1. Preprocessor: add the I/O library    */

int main()              /* 2. main(): program starts here          */
{                       /* 3. Opening brace: block begins          */
    int a = 10, b = 20; /* 4. Declaration + initialization         */
    int sum;
    sum = a + b;        /* 5. Statement (ends with a semicolon)    */
    printf("Sum = %d\n", sum);  /* 6. Output                       */
    return 0;           /* 7. Tell OS: finished successfully       */
}                       /* 8. Closing brace: block ends            */
```

OUTPUT

```
Sum = 30
```

- **Comments** — `// single line` and `/* multi line */`; ignored by the compiler.
- **Keywords** are reserved words (`int, if, while, return`) — C has 32 of them.
- **Tokens** are the smallest units: keywords, identifiers, constants, strings, operators, punctuators.
- C is **case-sensitive**: `Sum` and `sum` are different. Every statement ends with **;**.

1.5 Algorithms, Flowcharts (Dia Tool) & Pseudocode

◆ Definition

An **algorithm** is a finite, step-by-step, unambiguous list of instructions to solve a problem. **Pseudocode** writes the algorithm in plain English with programming-like structure. A **flowchart** draws the algorithm using standard symbols.

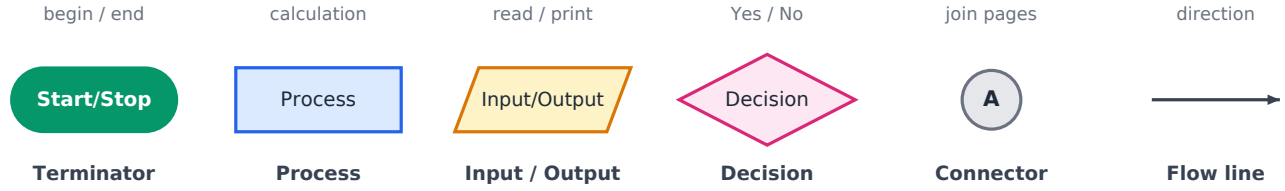
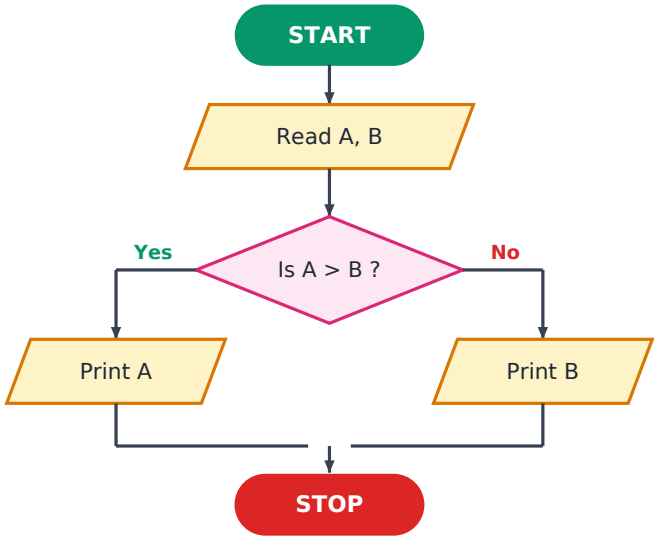


Fig 1.5 — Standard flowchart symbols



PSEUDOCODE

```
Algorithm: Largest of two numbers
1. START
2. READ A, B
3. IF A > B THEN
4.       PRINT A
5. ELSE
6.       PRINT B
7. END IF
8. STOP
```

Fig 1.6 — Flowchart + pseudocode: largest of two numbers

Drawing flowcharts with the Dia tool

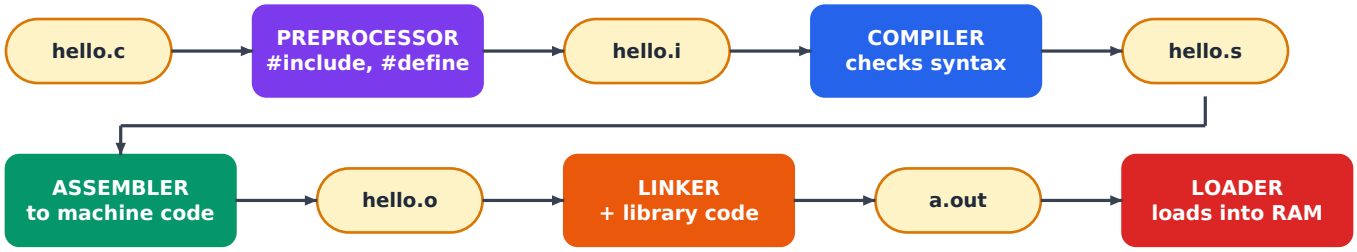
Dia is a free diagram editor used to draw flowcharts. Steps:

- **Open Dia** → in the toolbox drop-down choose the sheet **"Flowchart"**.
- **Drag symbols** (Terminal, Box, Diamond, Parallelogram) onto the canvas; **double-click** a symbol to type text.
- Select the **Line / Arrow** tool and join the symbols; connection points turn **red** when attached.
- Align with *Objects → Align*, then save as `.dia` and **File → Export** as PNG / PDF / SVG.

★ Exam Focus
Rules of flowcharting: one START and one STOP; flow runs top-to-bottom / left-to-right; a decision has **one input and two outputs** (Yes/No); every symbol has meaningful text; no crossing lines.

1.6 Introduction to Compilation and Execution

A computer cannot run C source code directly. It goes through **four stages** before it runs:



Source code → Object code → Executable → Running program
Fig 1.7 — Compilation and execution process in C (GCC)

Stage	Job
Preprocessing	Removes comments, expands `#include` and `#define`
Compilation	Checks syntax; converts to assembly. **Syntax errors** appear here
Assembling	Converts assembly into machine-level **object file** (.o / .obj)
Linking	Joins object file with library functions (e.g. printf) to make the **executable**

Terminal

```
gcc hello.c -o hello     # compile + link  (creates executable "hello")
./hello                  # run on Linux / macOS    (Windows: hello.exe)
gcc -Wall hello.c        # compile with all warnings switched on
```

Error type	When	Example
Syntax error	Compile time	Missing semicolon, misspelt keyword
Linker error	Link time	Calling a function that is not defined
Runtime error	While running	Divide by zero, invalid memory access
Logical error	Wrong output	Using + instead of * (program runs but result is wrong)

1.7 Primitive Data Types, Variables and Constants

A **data type** tells the compiler how much memory to reserve and what kind of value is stored.

Type	Size	Range (typical)	Format	Example
`char`	1 byte	−128 to 127	`%c`	`char g = 'A';`
`short`	2 bytes	−32,768 to 32,767	`%hd`	`short s = 100;`
`int`	4 bytes	-2.14×10^9 to 2.14×10^9	`%d`	`int age = 20;`
`long`	4 / 8 bytes	larger than int	`%ld`	`long pop = 9000000L;`
`unsigned int`	4 bytes	0 to 4,294,967,295	`%u`	`unsigned int n = 5;`
`float`	4 bytes	≈ 6–7 decimal digits	`%f`	`float pi = 3.14f;`
`double`	8 bytes	≈ 15–16 decimal digits	`%lf`	`double e = 2.71828;`
`void`	0	no value	–	`void f(void)`

Variables

A **variable** is a named memory location whose value can change. **Declare before use**:

Variables

```c
int age;              // declaration  (memory reserved, value = garbage)
age = 18;             // assignment
float marks = 87.5f;  // declaration + initialization
int x = 5, y = 10;    // multiple variables
```

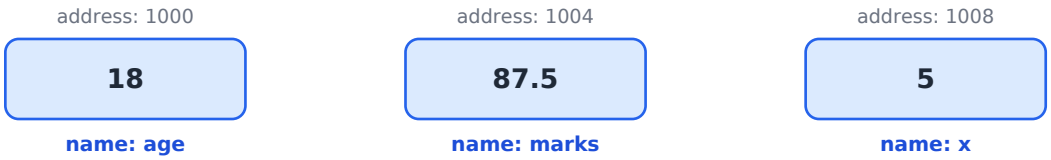


Fig 1.8 — A variable = name + address + value (box in memory)

- **Rules for identifier names:** letters, digits, underscore only; must **not start with a digit**; no spaces or special symbols; cannot be a keyword. Valid: `total_1, _x`. Invalid: `1st, my-var, int`.

Constants

Constants

```c
const float PI = 3.14159;   // const keyword: value cannot change
#define MAX 100             // symbolic constant (macro, no semicolon!)
// Literals:  10 (int)  3.5 (double)  'A' (char)  "Hi" (string)
```

1.8 Basic Input and Output

C uses `printf()` to print and `scanf()` to read. Both need `#include <stdio.h>`.

Specifier	Prints	Specifier	Prints
`%d / %i`	int	`%c`	single character
`%f`	float	`%s`	string
`%lf`	double	`%u`	unsigned int
`%.2f`	float with 2 decimals	`%x / %o`	hexadecimal / octal
`\n \t \\`	new line, tab, backslash	`%5d`	width 5, right aligned

Input & output

```c
#include <stdio.h>
int main() {
    int age; float h; char grade;
    printf("Enter age, height, grade: ");
    scanf("%d %f %c", &age, &h, &grade);
    printf("Age=%d  Height=%.1f  Grade=%c\n", age, h, grade);
    return 0;
}
```

OUTPUT

```
Enter age, height, grade: 19 5.8 A
Age=19  Height=5.8  Grade=A
```

> ⚠ **Common Mistake**
> Forgetting **&** in `scanf("%d", age)` — the program crashes. Use `%d` for int, `%f` for float. For single characters you can also use `getchar()` and `putchar()`; for strings with spaces use `fgets()`.

1.9 Operations (Operators) in C

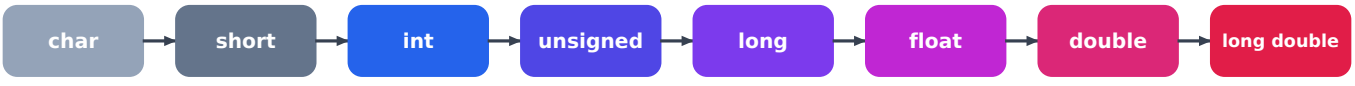
Category	Operators	Example (a=7, b=2)	Result
Arithmetic	+ − * / %	a / b , a % b	3 , 1 (integer division!)
Relational	< > <= >= == !=	a > b	1 (true)
Logical	&& || !	(a>5) && (b>5)	0 (false)
Assignment	= += −= *= /= %=	a += 3	a becomes 10
Increment / Decrement	++ --	a++ (post), ++a (pre)	a becomes 8
Conditional (ternary)	? :	max = (a>b) ? a : b;	max = 7
Bitwise	& | ^ ~ << >>	a & b , a << 1	2 , 14
sizeof	sizeof(type)	sizeof(int)	4

Priority	Operators (high → low)	Associativity
1	( ) ++ -- (postfix)	Left to right
2	! ~ ++ -- (prefix) sizeof, (type) cast	Right to left
3	* / %	Left to right
4	+ −	Left to right
5	< <= > >= then == !=	Left to right
6	&& then ||	Left to right
7	?: then = += −= ...	Right to left

> ✔ **Remember**
> **Post vs Pre:** if `a = 5`: `b = a++` gives **b=5, a=6** (use, then increase); `b = ++a` gives **b=6, a=6** (increase, then use). Example: `2 + 3 * 4 = 14` (multiplication first).

1.10 Type Conversion and Casting

When different data types are mixed in an expression, C converts them. There are two kinds:



smaller / lower rank — **automatic promotion →** — bigger / higher rank

Fig 1.9 — Implicit conversion: lower type is promoted to the higher type

	Implicit (automatic)	Explicit (type casting)
Done by	Compiler	Programmer
Syntax	None	`(type) expression`
Direction	Lower → higher (safe)	Any, may lose data

Conversion & casting

```c
int a = 5, b = 2;
float r1 = a / b;          // r1 = 2.0   (integer division first!)
float r2 = (float)a / b;   // r2 = 2.5   (a cast to float -> float division)
int   n  = 3.99;           // n = 3      (fraction truncated)
char  c  = 'A' + 1;        // c = 'B'    (ASCII 65 + 1 = 66)
```

OUTPUT

```
r1 = 2.0
r2 = 2.5
n  = 3
c  = B
```

1.11 Problem Solving Techniques

Algorithmic approach — the 6 steps

1 Understand the problem → **2 Plan** (inputs, outputs, process) → **3 Design** the algorithm / flowchart → **4 Code** in C → **5 Test & debug** → **6 Document & maintain**.

Characteristics of a good algorithm

Property	Meaning
Input	Zero or more well-defined inputs
Output	At least one output (the result)
Definiteness	Every step is clear and unambiguous
Finiteness	Must terminate after a finite number of steps
Effectiveness	Each step is simple enough to be carried out exactly (even by hand)
Correctness / Generality	Gives right answer for all valid inputs

Problem-solving strategies

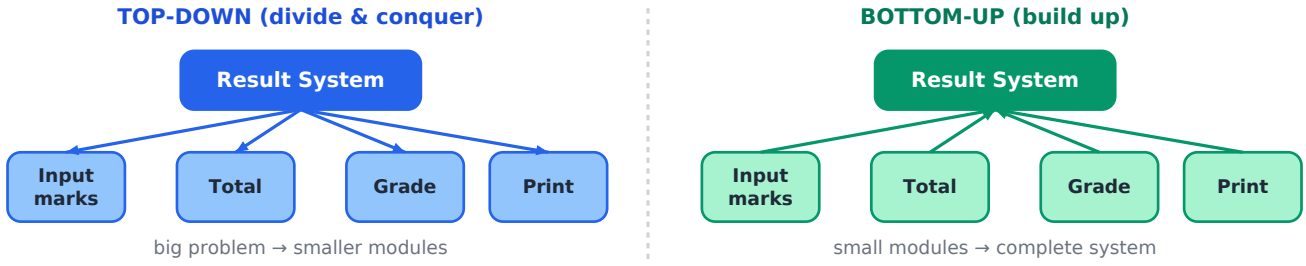


Fig 1.10 — Top-down vs bottom-up design

	Top-down approach	Bottom-up approach
Starts from	Whole problem, then breaks it into sub-problems	Small basic modules, then combines them
Used in	Procedural languages like C (functions)	Object-oriented design, reusing existing library code
Advantage	Easy to plan, understand and debug	Code reuse; modules can be tested early
Drawback	Needs full problem knowledge first	Overall design may be unclear at first

Time and space complexity of algorithms

Time complexity = how the running time grows with input size **n**. **Space complexity** = how the extra memory grows with **n**. We express both using **Big-O notation** (worst-case upper bound), ignoring constants.

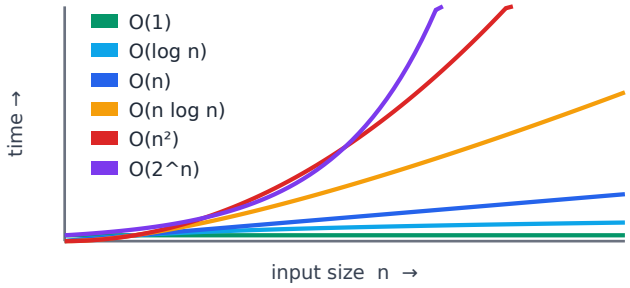


Fig 1.11 — Growth rate of common complexities

EXAMPLES

```
// O(1): one step
x = arr[0];

// O(n): loop runs n times
for (i = 0; i < n; i++)
    sum += i;

// O(n^2): nested loops
for (i = 0; i < n; i++)
  for (j = 0; j < n; j++)
    count++;
```

Big-O	Name	Example algorithm
O(1)	Constant	Access an array element
O(log n)	Logarithmic	Binary search
O(n)	Linear	Linear search, finding max
O(n log n)	Linearithmic	Merge sort
$O(n^2)$	Quadratic	Bubble sort, matrix addition
$O(2^n)$	Exponential	Recursive Fibonacci

★ Exam Focus

Space example: swapping two numbers using a temp variable needs 1 extra variable → **O(1)** space. Copying an array of n elements needs n extra cells → **O(n)** space. Rule: drop constants and lower-order terms — $3n^2 + 5n + 2$ becomes **$O(n^2)$**.

1.12 Solved Examples and Important Formulas

Problem	Formula	C expression
Area of circle	$A = \pi r^2$	`area = PI * r * r;`
Simple interest	SI = (P × R × T) / 100	`si = (p * r * t) / 100;`
Celsius → Fahrenheit	F = C × 9/5 + 32	`f = c * 9 / 5 + 32;`
Average of 3 numbers	Avg = (a + b + c) / 3	`avg = (a + b + c) / 3.0;`
Seconds → h : m : s	h = s/3600, m = (s%3600)/60, sec = s%60	`h = s / 3600;`

Sequential programs (no decisions, no loops)

```
#include <stdio.h>
#define PI 3.14159
int main() {
    float r = 5, area, P = 1000, R = 8, T = 2, si, c = 37, f;
    int a = 10, b = 20, temp, s = 3725;
    area = PI * r * r;            // area of circle
    si   = (P * R * T) / 100;     // simple interest
    f    = c * 9 / 5 + 32;        // temperature
    printf("Area=%.2f  SI=%.1f  F=%.1f\n", area, si, f);
    temp = a;  a = b;  b = temp;  // swap using a third variable
    printf("a=%d  b=%d\n", a, b);
    printf("%dh %dm %ds\n", s / 3600, (s % 3600) / 60, s % 60);
    return 0;
}
```

OUTPUT

```
Area=78.54  SI=160.0  F=98.6
a=20  b=10
1h 2m 5s
```

★ UNIT SUMMARY — QUICK REVISION

- A computer = **Input → CPU (ALU + CU + registers) → Output** with memory; the **PC** holds the next instruction's address.
- Languages: machine → assembly → high-level; translators are **assembler, compiler, interpreter**.
- C program: `#include`, `main()`, declarations, statements, `return 0;`
- Algorithm → **flowchart / pseudocode** → code → **preprocess → compile → assemble → link → run**.
- Data types: **char, int, float, double**; constants via `const` or `#define`; I/O via `printf / scanf` (use **&**).
- Remember: **int / int = int**; use a **cast** for decimals. Precedence: ( ) → * / % → + − → relational → logical → =
- Algorithm traits: input, output, **definiteness, finiteness, effectiveness**. Complexity: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.

Q Exam-Oriented Questions

Short Answer (2 marks)

1. Define a computer. List its basic units.
2. What is the role of the Program Counter?
3. Differentiate between a compiler and an interpreter.
4. What is an algorithm? List any four characteristics.
5. Name any four flowchart symbols and their uses.
6. What is the difference between `const` and `#define`?
7. Why is `&` used in `scanf()`?
8. What is type casting? Give an example.
9. Distinguish top-down and bottom-up approaches.
10. What is the time complexity of linear search and binary search?

Long Answer / Programs (5–10 marks)

1. Explain the block diagram of a computer with a neat sketch.
2. Describe the generations of computers with their technologies.
3. Explain the phases of compilation and execution of a C program with a diagram.
4. Write an algorithm, pseudocode and flowchart to find the largest of three numbers.
5. Explain all C data types with size, range and format specifier.
6. Explain the types of operators in C with examples and the precedence table.
7. Explain implicit and explicit type conversion with examples.
8. Explain Big-O notation. Find the time complexity of nested loops and a single loop.

INTRODUCTION TO PROGRAMMING • DIGITAL NOTES

UNIT II

Control Structures

Sequential • if / if-else / switch • for / while / do-while • break & continue

▶ What you will learn in this unit

- Simple sequential programs
- if, if-else, else-if ladder, nested if
- switch-case statement
- Choosing between if and switch
- for loop
- while loop
- do-while loop
- Nested loops & patterns
- break and continue
- Classic programs (prime, factorial...)
- Common mistakes
- Exam questions

2.1 Simple Sequential Programs

◆ Definition

Control structures decide the **order** in which statements run. There are three kinds: **Sequence** (one after another), **Selection** (choose a path: if, switch) and **Iteration** (repeat: loops).

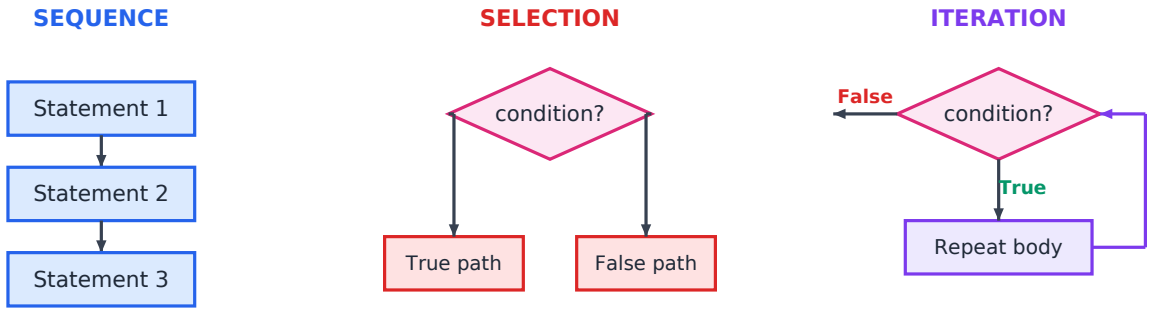


Fig 2.1 — The three basic control structures

A **sequential program** has no decisions or loops — statements run top to bottom. Pattern: **Input → Process → Output**.

Area & perimeter of a rectangle

```c
#include <stdio.h>
int main() {
    float l, b;
    printf("Enter length and breadth: ");
    scanf("%f %f", &l, &b);                    // INPUT
    printf("Area = %.2f\n", l * b);             // PROCESS + OUTPUT
    printf("Perimeter = %.2f\n", 2 * (l + b));
    return 0;
}
```

OUTPUT

```
Enter length and
breadth: 4 5
Area = 20.00
Perimeter = 18.00
```

How to write any program: IPO + dry run

- **Step 1 – Input:** what must the user give? (declare variables)
- **Step 2 – Process:** which formula / decision / repetition turns input into result?
- **Step 3 – Output:** what must be displayed and in what format?
- **Step 4 – Dry run:** test the program on paper with sample values (a **trace table**) before running it.

Celsius to Fahrenheit

```c
float c, f;
printf("Enter temperature in Celsius: ");
scanf("%f", &c);
f = (c * 9 / 5) + 32;                 // formula from the problem
printf("Fahrenheit = %.1f\n", f);
```

OUTPUT

```
Enter temperature in
Celsius: 100
Fahrenheit = 212.0
```

2.2 Conditional Statements

Conditions use relational (`< > == !=`) and logical (`&& || !`) operators. In C, **0 is false** and **any non-zero value is true**.

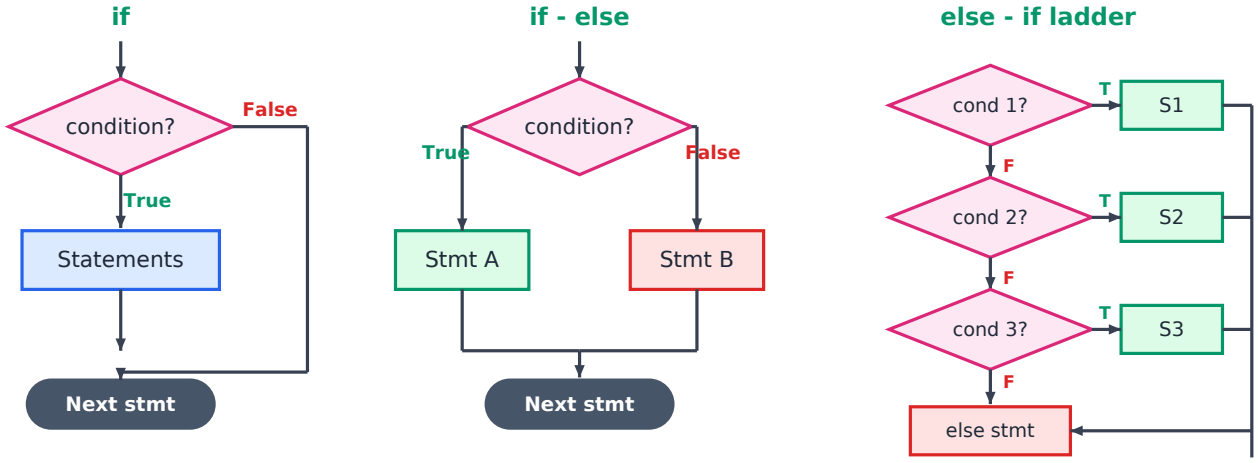


Fig 2.2 — Flowcharts of if, if-else and else-if ladder

Syntax of if family

```c
if (marks >= 40)                  // simple if
    printf("Pass");

if (age >= 18) printf("Adult");   // if-else
else           printf("Minor");

if      (m >= 90) printf("A");    // else-if ladder: first TRUE wins
else if (m >= 75) printf("B");
else if (m >= 60) printf("C");
else              printf("Fail");
```

Nested if — largest of three

```c
int a = 12, b = 45, c = 30, max;
if (a >= b) {                          // NESTED if: an if inside an if
    if (a >= c) max = a; else max = c;
} else {
    if (b >= c) max = b; else max = c;
}
printf("Largest = %d", max);
// Leap year:  (y%4==0 && y%100!=0) || (y%400==0)
```

OUTPUT

```
Largest = 45
```

> ⚠ **Common Mistake**
> `if (a = 5)` **assigns** 5 (always true). Use `==` to compare. Also never put a semicolon after the condition: `if (x > 0);` makes the body run always.

A	B	A && B (AND)	A || B (OR)	!A (NOT)
1 (true)	1 (true)	1	1	0
1 (true)	0 (false)	0	1	0
0 (false)	1 (true)	0	1	1
0 (false)	0 (false)	0	0	1

Conditional (ternary) operator

```
max = (a > b) ? a : b;     // ternary operator = compact if-else:  condition ? if_true : if_false
printf("%s", (n % 2 == 0) ? "Even" : "Odd");
```

switch statement

Use `switch` when one variable/expression is compared with many **constant** values (int or char).

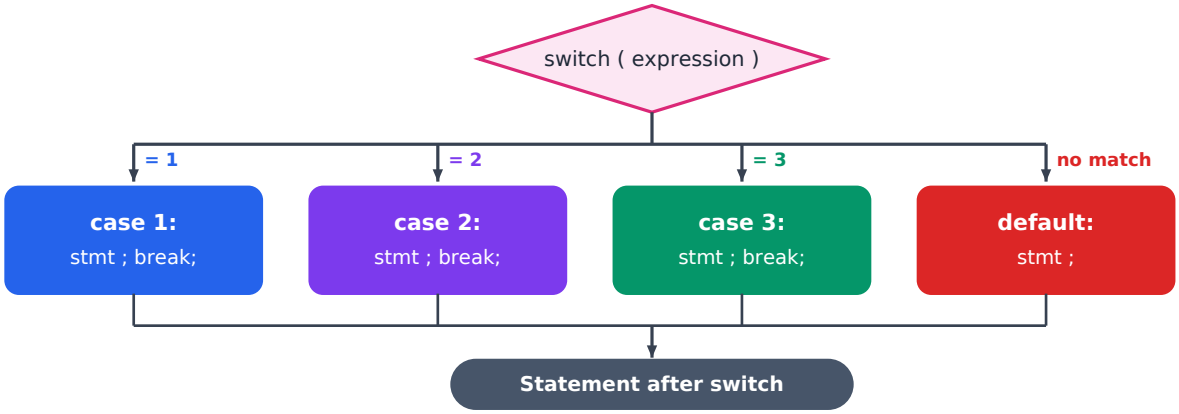


Fig 2.3 — Flow of the switch statement (break jumps out)

Calculator using switch

```
char op; float x, y;
scanf("%f %c %f", &x, &op, &y);
switch (op) {
    case '+': printf("%.2f", x + y); break;
    case '-': printf("%.2f", x - y); break;
    case '*': printf("%.2f", x * y); break;
    case '/': if (y != 0) printf("%.2f", x / y);
              else printf("Divide by zero");
              break;
    default : printf("Invalid operator");   // optional
}
```

OUTPUT

```
Input:  8 * 2.5
Output: 20.00
```

Grouping cases (vowel check)

```
switch (ch) {                  // no break = fall-through
    case 'a': case 'e': case 'i': case 'o': case 'u':
        printf("Vowel");  break;
    default:
        printf("Consonant");
}
```

OUTPUT

```
ch = 'e'  ->  Vowel
```

	if – else	switch
Tests	Ranges and complex conditions (`x > 5 && y < 3`)	Only equality with constants
Data types	Any (int, float, ...)	int and char only (not float / string)
Speed / clarity	Slower for many choices	Cleaner and faster for menus
Default branch	`else`	`default` (optional)

★ Exam Focus
Rules: case labels must be **unique constants**; `break` is needed to stop fall-through; `default` may appear anywhere; switch cannot test ranges or floating-point values.

2.3 Loops (Iteration)

◆ Definition
A **loop** repeats a block of statements while a condition stays true. Every loop has: **1) initialization, 2) condition, 3) update, 4) body**.

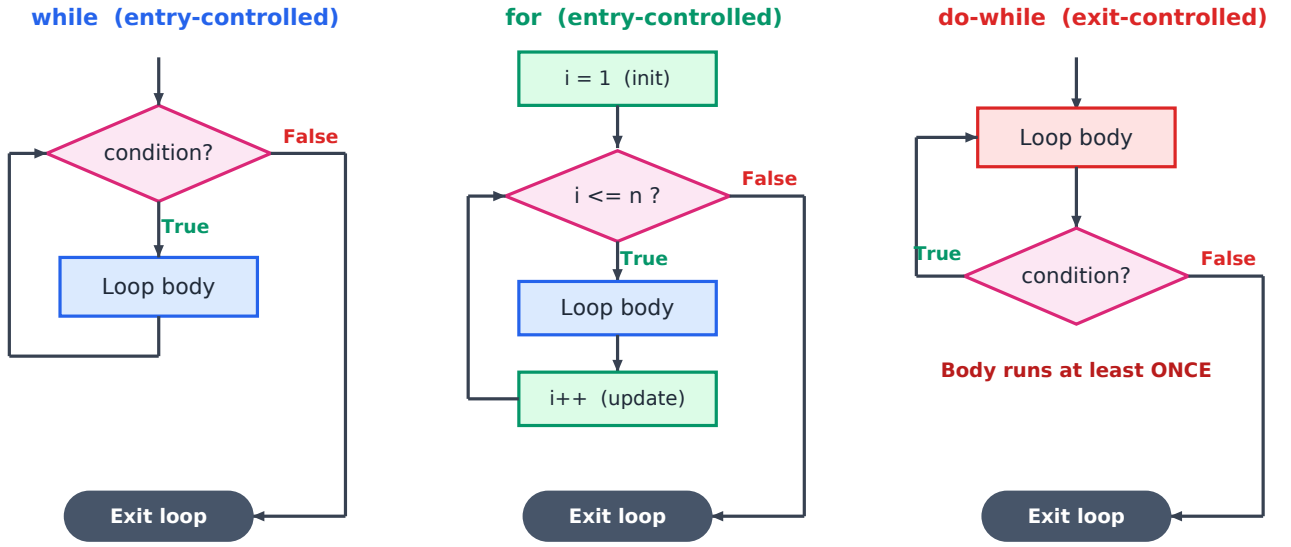


Fig 2.4 — Flowcharts of the three loops

Syntax of loops

```
for (init; condition; update)       // best when the number of repetitions is KNOWN
    body;

init;                               // while: repetitions UNKNOWN, test first
while (condition) { body; update; }

init;                               // do-while: body first, test later (menus, validation)
do { body; update; } while (condition);     // <- note the semicolon!
```

for (init ; condition ; update) { body } — order of execution: 1 → 2 → 4 → 3 → 2 → 4 → 3 … → 2 (false) → exit

1	2	3	4
INITIALIZE `int i = 1`	**CONDITION** `i <= 5`	**UPDATE** `i++`	**BODY** `{ sum += i; }`
once, at start	checked every time	after each pass	runs if true

Fig 2.5 — Anatomy of a for loop

i	Condition i<=5	sum before	sum after (sum += i)
1	true	0	1
2	true	1	3
3	true	3	6
4	true	6	10
5	true	10	15
6	**false → exit**	15	15 (final)

Sum of first 5 numbers (for)

```c
int i, sum = 0;
for (i = 1; i <= 5; i++)       // trace table above
    sum += i;
printf("Sum = %d", sum);
```

OUTPUT

```
Sum = 15
```

while loop

```c
int n = 5, i = 1;
while (i <= 5) {                        // multiplication table of 5
    printf("%d x %d = %d\n", n, i, n * i);
    i++;                                // forgetting this = infinite loop!
}
```

OUTPUT

```
5 x 1 = 5
5 x 2 = 10
...
5 x 5 = 25
```

do-while — input validation

```c
int n;
do {
    printf("Enter n > 0: ");
    scanf("%d", &n);
} while (n <= 0);              // repeat until input is valid
```

OUTPUT

```
Enter n > 0: -4
Enter n > 0: 7
```

Feature	for	while	do-while
Type	Entry-controlled	Entry-controlled	Exit-controlled
Condition checked	Before each pass	Before each pass	**After** each pass
Minimum runs	0	0	**1**
Best used when	Count is known	Count is unknown	Must run at least once (menu)
Semicolon at end	No	No	**Yes**

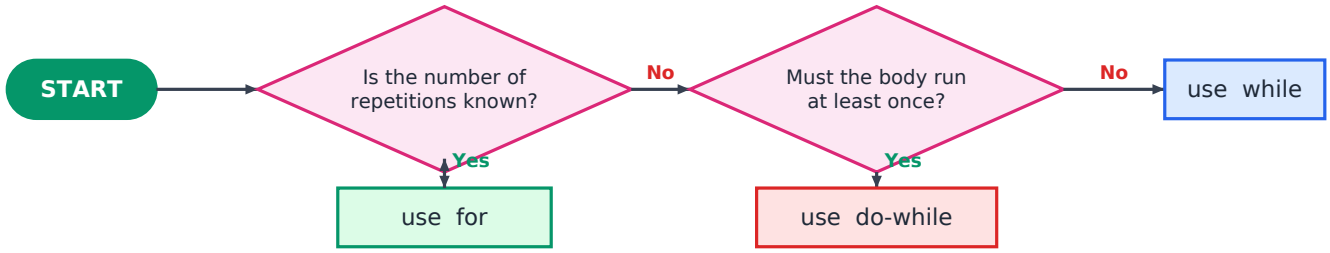


Fig 2.6 — Which loop should I use?

Infinite loops

A loop that never ends: `while(1)`, `for(;;)`, or a condition that never becomes false (missing update). Stop it with `break` or **Ctrl + C**.

Nested loops and patterns

A loop inside another loop: the **inner loop runs fully for every single pass of the outer loop**. Outer loop = rows, inner loop = columns. Total runs = outer × inner.

Star triangle

```c
for (i = 1; i <= 4; i++) {          // rows
    for (j = 1; j <= i; j++)        // columns: i stars in row i
        printf("* ");
    printf("\n");                   // new line after each row
}
```

OUTPUT

```
*
* *
* * *
* * * *
```

Outer i	Inner j runs	Printed in the row	Inner body count
1	j = 1	*	1
2	j = 1, 2	* *	2
3	j = 1, 2, 3	* * *	3
4	j = 1, 2, 3, 4	* * * *	4 → total = 1+2+3+4 = **10**

Pyramid

```c
for (i = 1; i <= 3; i++) {
    for (s = 1; s <= 3 - i; s++) printf(" ");        // leading spaces
    for (j = 1; j <= 2 * i - 1; j++) printf("*");    // stars
    printf("\n");
}
```

OUTPUT

```
  *
 ***
*****
```

More patterns

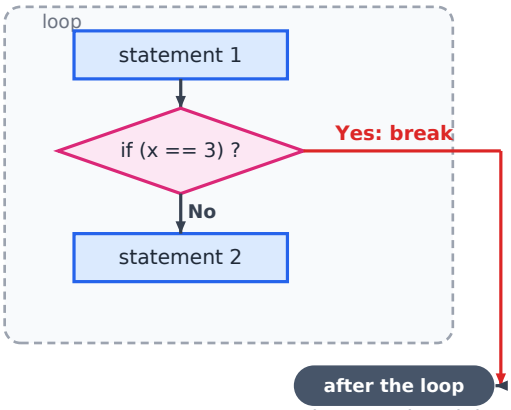
```c
for (i = 1; i <= 4; i++) {            // number triangle
    for (j = 1; j <= i; j++) printf("%d ", j);
    printf("\n");
}
for (i = 3; i >= 1; i--) {            // inverted star triangle
    for (j = 1; j <= i; j++) printf("* ");
    printf("\n");
}
```

OUTPUT

```
1
1 2
1 2 3
1 2 3 4
* * *
* *
*
```

2.4 break and continue

break — exit the loop completely



continue — skip rest, start next pass

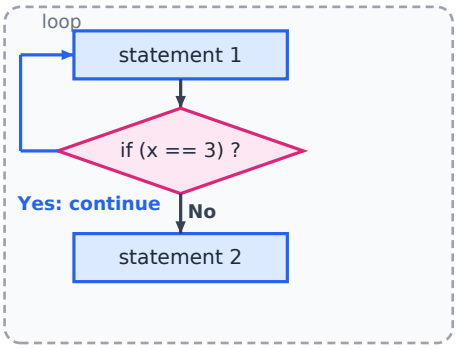


Fig 2.7 — break leaves the loop; continue jumps to the next iteration

break

```c
for (i = 1; i <= 10; i++) {
    if (i == 6) break;          // stop the whole loop at 6
    printf("%d ", i);
}
```

OUTPUT

```
1 2 3 4 5
```

continue

```c
for (i = 1; i <= 5; i++) {
    if (i == 3) continue;       // skip only i = 3
    printf("%d ", i);
}
```

OUTPUT

```
1 2 4 5
```

	break	continue
Effect	Terminates the nearest loop / switch	Skips the rest of the current pass
Control goes to	First statement after the loop	Loop update / condition check
Used in	Loops and **switch**	Loops only
Typical use	Search found, invalid input, prime test	Skip unwanted values

2.5 Classic Programs Using Control Structures

Factorial

```c
long fact = 1; int n = 5, i;
for (i = 1; i <= n; i++) fact *= i;          // 5! = 1x2x3x4x5
printf("%d! = %ld", n, fact);
```

OUTPUT

```
5! = 120
```

Fibonacci series (8 terms)

```c
int a = 0, b = 1, c, i;
for (i = 1; i <= 8; i++) {
    printf("%d ", a);
    c = a + b;  a = b;  b = c;          // add previous two
}
```

OUTPUT

```
0 1 1 2 3 5 8 13
```

Digit sum and reverse

```c
int n = 1234, rev = 0, sum = 0, digit;
while (n > 0) {
    digit = n % 10;            // last digit
    sum  += digit;
    rev   = rev * 10 + digit;  // build reversed number
    n     = n / 10;            // drop last digit
}
printf("Sum=%d Reverse=%d", sum, rev);   // palindrome if n == rev
```

OUTPUT

```
Sum=10 Reverse=4321
```

Prime number check

```c
int n = 29, i, prime = 1;
if (n < 2) prime = 0;
for (i = 2; i * i <= n; i++)            // test divisors up to sqrt(n)
    if (n % i == 0) { prime = 0; break; }   // divisor found -> not prime
printf(prime ? "Prime" : "Not prime");
```

OUTPUT

```
Prime
```

GCD of two numbers

```c
int a = 48, b = 18, t;
while (b != 0) {              // Euclid: replace (a,b) by (b, a mod b)
    t = b;  b = a % b;  a = t;
}
printf("GCD = %d", a);
```

OUTPUT

```
GCD = 6
```

Armstrong number (3 digits)

```c
int n = 153, t = n, dg, s = 0;
while (t > 0) { dg = t % 10;  s += dg * dg * dg;  t /= 10; }
printf(s == n ? "Armstrong" : "Not Armstrong");   // 1+125+27 = 153
```

OUTPUT

```
Armstrong
```

Menu-driven program (do-while + switch)

```c
int ch;
do {
    printf("1.Add 2.Sub 3.Exit : ");
    scanf("%d", &ch);
    switch (ch) {
        case 1: printf("Add selected\n"); break;
        case 2: printf("Sub selected\n"); break;
        case 3: printf("Bye!\n");         break;
        default: printf("Invalid\n");
    }
} while (ch != 3);
```

OUTPUT

```
1.Add 2.Sub 3.Exit : 1
Add selected
1.Add 2.Sub 3.Exit : 3
Bye!
```

● Real-life Use

Loops power menus, ATMs (repeat until Exit), games (game loop), searching/sorting data, billing totals and password retry limits. Selection statements power every login check, grade calculator and traffic-light controller.

Common mistakes at a glance

Mistake	Result	Fix
`if (x = 5)`	Assigns, always true	Use `==`
`for (...);` or `if (...);`	Empty body, next line runs once	Remove the extra **;**
Missing `break` in switch	Fall-through to next cases	Add `break;`
Missing update (`i++`)	Infinite loop	Always change the loop variable
Off-by-one: `<` vs `<=`	One extra / missing pass	Trace first and last value

Predict the output (practice)

#	Code	Output	Why?
1	`for (i=0; i<3; i++) printf("%d ", i*i);`	0 1 4	i takes 0,1,2 and prints squares
2	`int i=10; do { printf("%d ", i); i--; } while (i>8);`	10 9	body first; stops when i = 8
3	`switch(2){ case 1: printf("One"); case 2: printf("Two"); case 3: printf("Three"); break; }`	TwoThree	no break after case 2 → falls through
4	`for (i=1; i<=5; i++) { if (i%2==0) continue; printf("%d", i); }`	135	even values skipped
5	`int x = 0; if (x = 5) printf("Yes");`	Yes	= assigns 5 (non-zero = true)
6	`printf("%d", (5 > 3) && (2 > 4));`	0	true AND false = false (0)
7	`int i=1; while (i<=3) printf("%d ", i);`	infinite 1 1 1 ...	i is never incremented

★ UNIT SUMMARY — QUICK REVISION

- **Sequence, selection, iteration** are the three control structures. In C, **0 = false**, non-zero = true.
- Selection: **if, if-else, else-if ladder, nested if** for conditions; **switch** for many constant choices (needs `break`).
- **for** (count known), **while** (test first), **do-while** (runs at least once; ends with **;**).
- Nested loop: inner loop completes for each outer pass → used for patterns, matrices, tables.
- **break** exits the loop/switch; **continue** skips to the next iteration.
- Digit tricks: `n % 10` = last digit, `n / 10` = remove last digit. Prime test needs divisors only up to √n.

Q Exam-Oriented Questions

Short Answer (2 marks)

1. Name the three basic control structures with one example each.
2. Differentiate between `=` and `==`.
3. What is an else-if ladder? When is it used?
4. What happens if `break` is missing in a switch case?
5. List two differences between while and do-while.
6. What is an infinite loop? Give a code example.
7. What is a nested loop? How many times does the inner statement run for outer=3, inner=4?
8. Differentiate between break and continue.
9. Can a switch use a float variable? Why?
10. Write the syntax of the for loop and explain its parts.

Long Answer / Programs (5–10 marks)

1. Explain all forms of the if statement with flowcharts and examples.
2. Explain the switch statement with syntax, flowchart and a menu-driven calculator program.
3. Compare for, while and do-while loops with flowcharts and programs.
4. Write programs: (a) leap year, (b) largest of three numbers, (c) grade from marks using else-if ladder.
5. Write programs: (a) factorial, (b) Fibonacci series, (c) prime check, (d) palindrome number.
6. Write a program to print the pyramid and the right-angled star triangle for n rows.
7. Explain break and continue with examples and a flowchart.
8. Trace the output: `for(i=1;i<=3;i++) for(j=1;j<=i;j++) printf("%d",j);`

INTRODUCTION TO PROGRAMMING • DIGITAL NOTES

UNIT III

Arrays and Strings

Indexing • Memory model • 1-D arrays • 2-D arrays • Strings & string functions

▶ What you will learn in this unit

- Why we need arrays
- Declaration & initialization
- Array indexing
- Memory model of arrays
- Programs on integer arrays
- Searching & sorting basics
- Two-dimensional arrays
- Matrix programs
- Introduction to strings
- String input / output
- String library functions
- Common mistakes & questions

3.1 Arrays: Concept, Declaration and Indexing

◆ Definition

An **array** is a collection of **same-type** elements stored in **continuous (adjacent) memory locations** and accessed with one name plus an **index**.

Without arrays, storing marks of 50 students needs 50 separate variables. With an array: `int marks[50];` — one name, 50 boxes, and a loop can process them all.

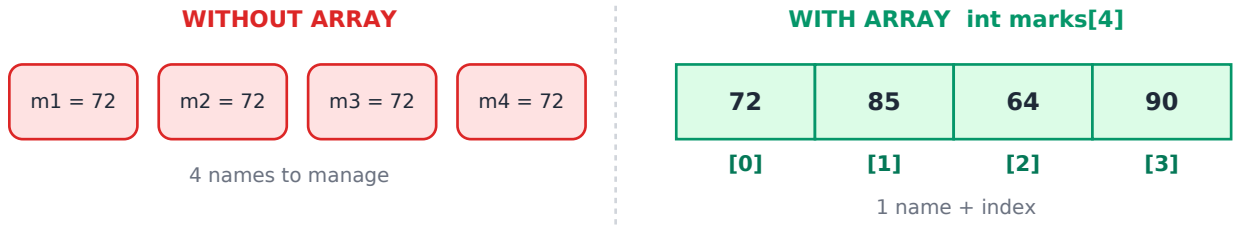


Fig 3.1 — Separate variables vs. one array

Task	Syntax	Example
Declaration	`type name[size];`	`int a[5];` (size must be a positive integer)
Initialization (full)	`type name[n] = {v1, v2, ...};`	`int a[5] = {10, 20, 30, 40, 50};`
Size from values	`type name[] = {...};`	`int a[] = {1, 2, 3};` → size 3
Partial	remaining become 0	`int a[5] = {1, 2};` → 1 2 0 0 0
All zeros	`= {0};`	`int a[100] = {0};`
Access / change	`name[index]`	`a[2] = 99; x = a[0];`

★ Exam Focus

Indexing starts at 0. An array of size **n** has valid indexes **0 to n−1**. The last element of `a[5]` is `a[4]`. C does **not** check bounds — `a[5]` or `a[-1]` silently reads/writes other memory (undefined behaviour).

Memory model of a 1-D array

Elements sit one after another. For `int a[5]` (4 bytes per int) starting at address 1000:

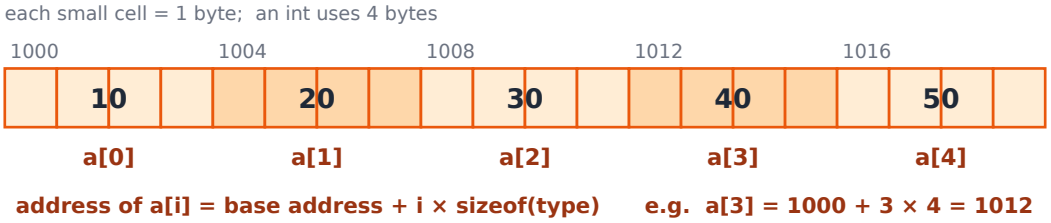


Fig 3.2 — Continuous memory layout of int a[5] (base address 1000)

✔ Remember
The array **name** itself is the **address of the first element** (`a == &a[0]`). This is why arrays and pointers are so closely related (Unit IV). Array size in bytes = `n × sizeof(type)`; number of elements = `sizeof(a) / sizeof(a[0])`.

3.2 Programs with Arrays of Integers

Traversing an array means visiting every element with a `for` loop from index **0** to **n−1**.

Read, sum and average

```c
int a[5], i, sum = 0;
printf("Enter 5 numbers: ");
for (i = 0; i < 5; i++) {          // reading: note the & before a[i]
    scanf("%d", &a[i]);
    sum += a[i];
}
printf("Sum = %d, Average = %.2f", sum, sum / 5.0);
```

OUTPUT

```
Enter 5 numbers:
10 20 30 40 50
Sum = 150, Average = 30.00
```

Largest and smallest

```c
int a[] = {42, 7, 56, 19, 33}, i, max, min;
max = min = a[0];                    // start with the first element
for (i = 1; i < 5; i++) {
    if (a[i] > max) max = a[i];
    if (a[i] < min) min = a[i];
}
printf("Max=%d Min=%d", max, min);
```

OUTPUT

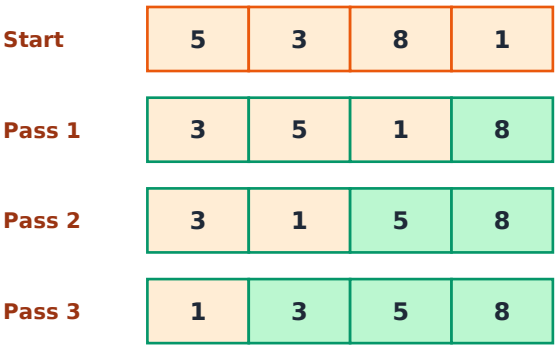
```
Max=56 Min=7
```

Linear search and reverse

```c
int key = 19, pos = -1;
for (i = 0; i < 5; i++)
    if (a[i] == key) { pos = i; break; }          // stop at first match
if (pos != -1) printf("Found at index %d", pos);
else           printf("Not found");
for (i = 4; i >= 0; i--) printf("%d ", a[i]);     // reverse printing
```

OUTPUT

```
Found at index 3
33 19 56 7 42
```



compare neighbours, swap if left > right

green = in final pla
biggest value "bubble" to the end each pass

BUBBLE SORT (ascending)

```c
for (i = 0; i < n-1; i++)
  for (j = 0; j < n-1-i; j++)
    if (a[j] > a[j+1]) {
      t = a[j];
      a[j] = a[j+1];   // swap
      a[j+1] = t;
    }
```

Fig 3.3 — Bubble sort on {5, 3, 8, 1}

Step	low	high	mid	a[mid]	Decision (key = 50)
1	0	6	3	40	40 < 50 → search right: low = mid+1
2	4	6	5	60	60 > 50 → search left: high = mid−1
3	4	4	4	50	**Found at index 4**

Binary search (trace table above)

```c
int a[] = {10, 20, 30, 40, 50, 60, 70};     // MUST be sorted
int lo = 0, hi = 6, mid, key = 50, pos = -1;
while (lo <= hi) {
    mid = (lo + hi) / 2;
    if      (a[mid] == key) { pos = mid; break; }
    else if (a[mid] <  key) lo = mid + 1;       // go right
    else                    hi = mid - 1;       // go left
}
```

OUTPUT

```
pos = 4
```

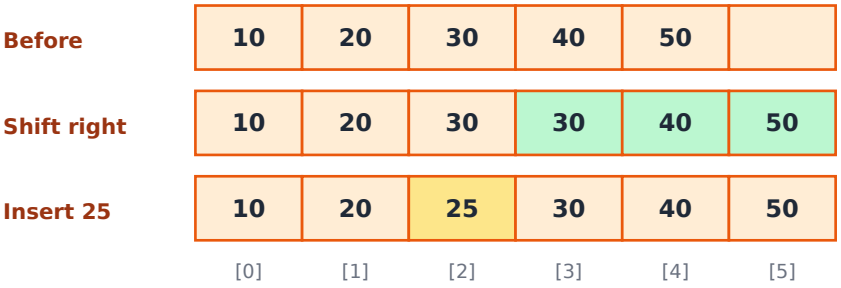


Fig 3.4 — Inserting 25 at position 2 (elements shift right, last element first)

INSERT / DELETE

```c
for (i = n-1; i >= pos; i--)
    a[i+1] = a[i]; // room
a[pos] = val;  n++;

// DELETE at pos:
for (i = pos; i < n-1; i++)
    a[i] = a[i+1]; // gap
n--;
```

Count even/odd and merge arrays

```c
int a[] = {42, 7, 56, 19, 33}, i, ev = 0, od = 0;
for (i = 0; i < 5; i++)
    if (a[i] % 2 == 0) ev++; else od++;          // even / odd test
printf("Even=%d Odd=%d\n", ev, od);
int A[] = {1, 3, 5}, B[] = {2, 4}, C[5];         // merge two arrays
for (i = 0; i < 3; i++) C[i] = A[i];
for (i = 0; i < 2; i++) C[3 + i] = B[i];         // continue after A
```

OUTPUT

```
Even=2 Odd=3
C = 1 3 5 2 4
```

Advantages of arrays	Limitations of arrays
Many values under one name; easy loop processing	Fixed size — cannot grow or shrink at run time
Fast direct access by index — O(1)	Insertion / deletion are slow (elements must shift)
Contiguous memory = cache friendly	All elements must be of the same type
Base for strings, matrices, stacks, queues	No bounds checking by the compiler

● Real-life Use

Marks lists, temperature logs, game boards (tic-tac-toe = 3×3 array), image pixels (2-D), spreadsheets, and storing text as character arrays.

Algorithm	Best	Worst	Needs sorted data?
Linear search	O(1)	O(n)	No
Binary search	O(1)	O(log n)	**Yes**
Bubble sort	O(n)	$O(n^2)$	—

3.3 Two-Dimensional Arrays

◆ Definition

A **2-D array** is an "array of arrays" — a table of **rows** and **columns**. Declared as `type name[ROWS][COLS];` and accessed as `name[row][col]`.

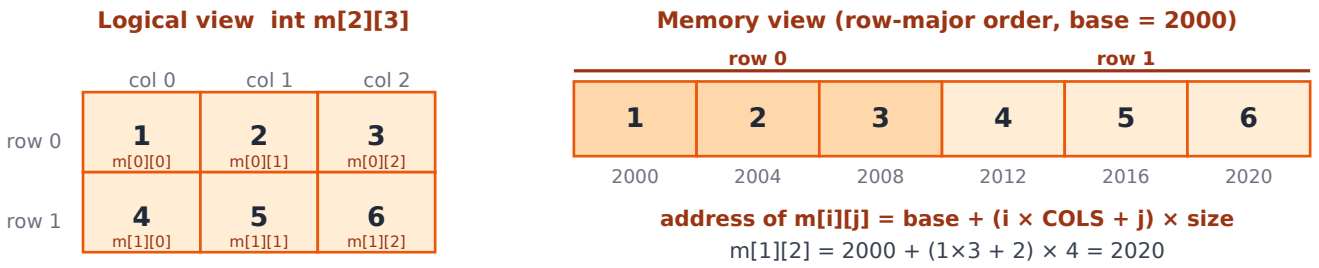


Fig 3.5 — A 2-D array in logical and memory view

Reading and printing a matrix

```c
int m[2][3] = {{1, 2, 3}, {4, 5, 6}};     // row by row
int i, j;
for (i = 0; i < 2; i++) {                  // outer loop = rows
    for (j = 0; j < 3; j++)                // inner loop = columns
        printf("%d ", m[i][j]);
    printf("\n");
}
```

OUTPUT

```
1 2 3
4 5 6
```

Matrix operations — all use nested loops:

Matrix addition and transpose

```c
for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++)
        C[i][j] = A[i][j] + B[i][j];      // ADDITION

for (i = 0; i < 2; i++)
    for (j = 0; j < 3; j++)
        T[j][i] = m[i][j];                // TRANSPOSE
```

OUTPUT

```
C =  6  8
    10 12

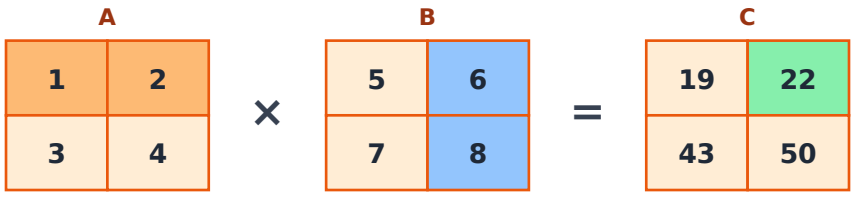
T =  1  4
     2  5
     3  6
```

Matrix multiplication C = A x B

```c
for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++) {
        C[i][j] = 0;
        for (k = 0; k < 2; k++)
            C[i][j] += A[i][k] * B[k][j];     // row of A  x  column of B
    }
```

OUTPUT

```
C = 19 22
    43 50
```



C[0][1] = (row 0 of A) · (col 1 of B)

= 1×6 + 2×8 = 6 + 16 = 22

orange = row of A, blue = column of B, green = resulting element

Fig 3.6 — How one element of the product matrix is computed

Row sums and diagonal sum of a 3x3 matrix

```c
int m[3][3] = {{1,2,3}, {4,5,6}, {7,8,9}}, i, j, rs, dsum = 0;
for (i = 0; i < 3; i++) {
    rs = 0;
    for (j = 0; j < 3; j++) rs += m[i][j];       // sum of row i
    printf("Row %d sum = %d\n", i, rs);
    dsum += m[i][i];                             // main diagonal
}
printf("Diagonal sum = %d", dsum);
```

OUTPUT

```
Row 0 sum = 6
Row 1 sum = 15
Row 2 sum = 24
Diagonal sum = 15
```

Operation	Rule	Result order	Loops
Addition / subtraction	same order; C[i][j] = A[i][j] ± B[i][j]	m × n	2
Transpose	T[j][i] = M[i][j]	n × m	2
Multiplication	cols of A = rows of B	m × p	3
3-D array	`int c[2][3][4];` = 2 tables of 3 rows × 4 columns	—	3

✔ Remember
Multiplication rule: A(m×n) × B(n×p) = C(m×p) — columns of A must equal rows of B. Example: C[0][0] = 1×5 + 2×7 = **19**. Addition needs both matrices of the **same order**.

3.4 Introduction to Strings

◆ Definition
A **string** is a one-dimensional **array of characters** that ends with the **null character** `\0` (ASCII 0). The `\0` tells functions where the string stops.

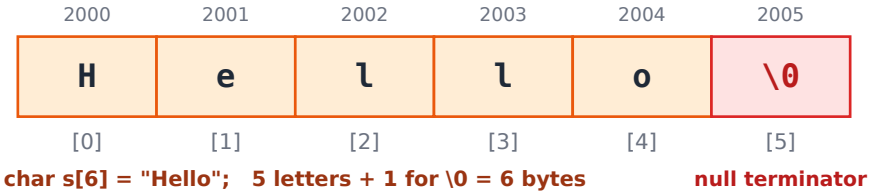


Fig 3.7 — A string in memory (each char = 1 byte)

Way	Code	Note
Literal	`char s[] = "Hi";`	size = 3 (H, i, \0), compiler adds \0
Fixed size	`char s[20] = "Hi";`	rest of array filled with \0
Char list	`char s[] = {'H','i','\0'};`	you must add \0 yourself
Pointer	`char *s = "Hi";`	string literal — cannot be modified

⚠ Common Mistake
"A" is a string (2 bytes: A and \0) but **'A'** is a single char (1 byte). Never compare strings with `==` or copy with `=`; use `strcmp()` and `strcpy()`.

Characters and ASCII values

Every character is stored as a number (ASCII code), so characters can be used in arithmetic:

Character	'0' … '9'	'A' … 'Z'	'a' … 'z'	space	'\n'	'\0'
ASCII code	48 … 57	65 … 90	97 … 122	32	10	0

Character arithmetic

```
char c = '7';
int digit = c - '0';        // 55 - 48 = 7  (char -> number)
char up = 'a' - 32;         // 97 - 32 = 65  -> 'A'      (or toupper('a'))
char nx = 'A' + 2;          // 65 + 2  = 67  -> 'C'
int diff = 'z' - 'a';       // 25
```

OUTPUT

```
digit = 7
up    = A
nx    = C
diff  = 25
```

String input and output

Function	Purpose	Remark
`scanf("%s", s)`	Reads one word	Stops at space; **no & needed**; risk of overflow
`fgets(s, size, stdin)`	Reads a full line	Safe; keeps the newline `\n`
`gets(s)`	Reads a line	**Unsafe, removed in C11** — avoid
`printf("%s", s)` / `puts(s)`	Prints string	`puts` adds a newline automatically
`getchar()` / `putchar(c)`	One character in / out	Used in character-by-character loops

Reading a line with spaces

```c
char name[30];
printf("Enter your full name: ");
fgets(name, 30, stdin);      // %s would stop at space
printf("Hello, %s", name);
```

OUTPUT

```
Enter your full name:
Asha Rao
Hello, Asha Rao
```

String library functions (#include <string.h>)

Function	What it does	Example	Result
`strlen(s)`	Length (without \0)	`strlen("Hello")`	5
`strcpy(d, s)`	Copy s into d	`strcpy(d, "Hi")`	d = "Hi"
`strcat(d, s)`	Append s to d	`strcat("Good", "Day")`	"GoodDay"
`strcmp(a, b)`	Compare: 0 if equal, <0 if a<b, >0 if a>b	`strcmp("apple","banana")`	negative
`strncpy / strncat`	Same, but limited to n chars (safer)	`strncpy(d, s, 3)`	copies 3 chars
`strchr(s, c) / strstr(s, t)`	Find a character / substring	`strstr("hello","ll")`	pointer to "llo"

ctype.h	Use	ctype.h	Use
`toupper(c)`	'a' → 'A'	`isalpha(c)`	letter?
`tolower(c)`	'A' → 'a'	`isdigit(c)`	digit 0-9?

Count vowels

```c
char s[] = "education";  int i, v = 0, ch;
for (i = 0; s[i] != '\0'; i++) {       // until null char
    ch = tolower(s[i]);
    if (ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u') v++;
}
printf("Vowels = %d, Consonants = %d", v, i - v);
```

OUTPUT

```
Vowels = 5,
Consonants = 4
```

Palindrome check

```c
char s[] = "madam";
int i = 0, j = strlen(s) - 1, ok = 1;
while (i < j) {                        // compare first with last, move inwards
    if (s[i] != s[j]) { ok = 0; break; }
    i++;  j--;
}
printf(ok ? "Palindrome" : "Not a palindrome");
```

OUTPUT

```
Palindrome
```

Length without strlen & array of strings

```c
char s[] = "Programming";  int len = 0;
while (s[len] != '\0') len++;                // own strlen
printf("Length = %d\n", len);
char days[3][10] = {"Mon", "Tue", "Wed"};   // array of strings
printf("%s", days[1]);
```

OUTPUT

```
Length = 11
Tue
```

string.h in action

```c
char a[20] = "Good", b[20];
strcpy(b, "Day");                // b = "Day"
strcat(a, " ");  strcat(a, b);  // a = "Good Day"
printf("%s (%d chars)\n", a, strlen(a));
printf("%d\n", strcmp("abc", "abc"));       // 0  -> equal
```

OUTPUT

```
Good Day (8 chars)
0
```

Sorting names alphabetically

```c
char n[3][10] = {"Ravi", "Amit", "Kiran"}, t[10];
for (i = 0; i < 2; i++)                          // bubble sort on strings
    for (j = 0; j < 2 - i; j++)
        if (strcmp(n[j], n[j+1]) > 0) {          // out of alphabetical order?
            strcpy(t, n[j]); strcpy(n[j], n[j+1]); strcpy(n[j+1], t);
        }
```

OUTPUT

```
Amit
Kiran
Ravi
```

Word count and uppercase

```c
char s[] = "hello world";  int i, words = 1, len = strlen(s);
for (i = 0; i < len; i++)
    if (s[i] == ' ') words++;        // a space = new word
for (i = 0; i < len; i++) s[i] = toupper(s[i]);
printf("%s has %d words", s, words);
```

OUTPUT

```
HELLO WORLD has
2 words
```

★ Exam Focus

Common errors: (1) forgetting space for `\0` → `char s[5] = "Hello"` overflows; (2) using `==` for strings; (3) accessing `a[n]`; (4) not using `&` with `scanf("%d", &a[i])` (but strings with `%s` need no &); (5) arrays cannot be assigned or compared directly: `a = b` is illegal.

3.5 More Solved Programs (Exam Favourites)

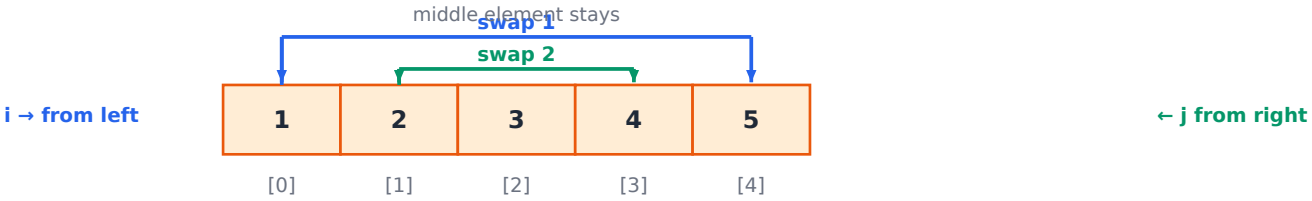


Fig 3.8 — Reversing an array in place: swap a[i] with a[j] until i ≥ j

Reverse in place and second largest

```c
int a[] = {1, 2, 3, 4, 5}, i, j, t;
for (i = 0, j = 4; i < j; i++, j--) {      // two indexes
    t = a[i];  a[i] = a[j];  a[j] = t;
}
// Second largest of {42, 7, 56, 19, 33}
int b[] = {42, 7, 56, 19, 33}, first = -1, second = -1;
for (i = 0; i < 5; i++) {
    if (b[i] > first)                  { second = first; first = b[i]; }
    else if (b[i] > second && b[i] != first)  second = b[i];
}
```

OUTPUT

```
a = 5 4 3 2 1
first  = 56
second = 42
```

Character frequency (counting array trick)

```c
char s[] = "banana";  int count[26] = {0}, i;
for (i = 0; s[i]; i++) count[s[i] - 'a']++;     // 'a'->0 ... 'z'->25
for (i = 0; i < 26; i++)
    if (count[i]) printf("%c=%d ", 'a' + i, count[i]);
```

OUTPUT

```
a=3 b=1 n=2
```

Symmetric matrix check

```c
int m[3][3] = {{1,2,3}, {2,5,6}, {3,6,9}}, sym = 1;
for (i = 0; i < 3; i++)
    for (j = 0; j < 3; j++)
        if (m[i][j] != m[j][i]) sym = 0;          // symmetric: m == transpose
printf(sym ? "Symmetric" : "Not symmetric");
```

OUTPUT

```
Symmetric
```

Task	Loop pattern	Remember
Visit every element	`for (i = 0; i < n; i++)`	last index is n − 1
Compare with the first / start from second	`max = a[0]; for (i = 1; ...)`	initialize with a real element
Compare neighbours (sorting)	`for (j = 0; j < n-1-i; j++)`	a[j+1] must stay inside the array
Work from both ends	`for (i=0, j=n-1; i < j; i++, j--)`	stop when the indexes meet
Every cell of a matrix	nested: outer = rows, inner = columns	m[i][j] → i row, j column
Walk through a string	`for (i = 0; s[i] != '\0'; i++)`	no need to know the length

★ UNIT SUMMARY — QUICK REVISION

- **Array** = same-type elements in **contiguous memory**; index runs **0 … n−1**; no bounds checking.
- Address of `a[i]` = **base + i × size**; the array name is the address of the first element.
- Process arrays with `for` loops: sum, max/min, linear search O(n), binary search O(log n) on sorted data, bubble sort O(n^2).
- **2-D array** `m[r][c]` is stored **row-major**; address = base + (i×COLS + j)×size; nested loops for matrix add, transpose, multiply.
- **String** = char array ending with **\0**; size needed = characters + 1. Use **fgets** for lines, **%s** for one word.
- `strlen, strcpy, strcat, strcmp` live in `<string.h>`; `strcmp` returns **0 when equal**.

Q Exam-Oriented Questions

Short Answer (2 marks)

1. Define an array. Why are arrays needed?
2. What are the valid indexes of `int a[10]`? What happens if `a[10]` is accessed?
3. How is a 1-D array stored in memory? Give the address formula.
4. Write the declaration of a 3×4 integer matrix and initialize it.
5. What is the role of the null character in a string?
6. Differentiate between `scanf("%s")` and `fgets()`.
7. Why can't two strings be compared using `==`?
8. What are the values returned by `strcmp()`?
9. Differentiate between `'A'` and `"A"`.
10. What is the time complexity of linear and binary search?

Long Answer / Programs (5–10 marks)

1. Explain declaration, initialization and accessing of 1-D arrays with examples.
2. Write programs to (a) find the largest and smallest, (b) search an element, (c) reverse an array.
3. Write a program to sort an array using bubble sort and show the pass-wise trace.
4. Explain 2-D arrays with memory layout. Write a program for matrix addition and multiplication.
5. Write a program to find the transpose of a matrix and the sum of its diagonal elements.

6. Explain any six string-handling functions with examples.
7. Write programs to (a) find the length of a string without strlen, (b) check a palindrome, (c) count vowels.
8. Write a program to read a line of text and convert it to uppercase.

INTRODUCTION TO PROGRAMMING • DIGITAL NOTES

UNIT IV

Pointers & User-Defined Data Types

Pointers • Address & dereference operators • Pointer arithmetic • Structures • Unions

IV

▶ What you will learn in this unit

- What is a pointer?
- Address (&) and dereference (*) operators
- Pointer declaration & NULL pointer
- Pointer to pointer
- Pointer & address arithmetic
- Array manipulation using pointers
- Structures: declare, access, nest
- Array of structures
- Pointer to structure (->) & typedef
- Unions & shared memory
- Structure vs union
- Common mistakes & questions

4.1 Pointers: Address and Dereference Operators

◆ Definition

A **pointer** is a variable that stores the **memory address** of another variable. Every variable lives at some address in RAM; a pointer lets you reach that variable **indirectly**.

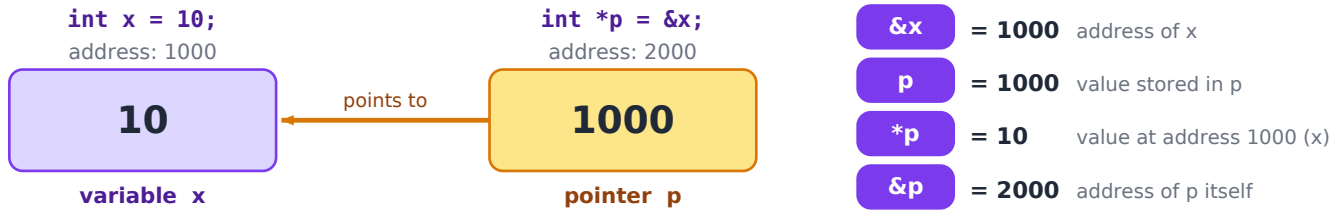


Fig 4.1 — A pointer holds the address of a variable

Operator	Name	Meaning	Example
`&`	Address-of	Gives the address of a variable	`p = &x;`
`*` (in declaration)	Pointer declaration	Creates a pointer variable	`int *p;`
`*` (in expression)	Dereference / indirection	Reads or writes the value at the address	`*p = 25; y = *p;`
`->`	Arrow	Member access through a struct pointer	`ps->marks`

Pointer basics

```c
int x = 10;
int *p = &x;                          // p stores the address of x
printf("x = %d, *p = %d\n", x, *p);   // both give 10
*p = 25;                              // change x via pointer
printf("x = %d\n", x);
printf("Address = %p\n", (void *)&x);  // %p = address
```

OUTPUT

```
x = 10, *p = 10
x = 25
Address = 0x7ffd...
```

- **Syntax:** `datatype *name;` — the data type says what kind of value lives at the address (`int *` → int, `char *` → char).
- **Size of a pointer** is the same for every type: 8 bytes on a 64-bit system (4 bytes on 32-bit).
- **NULL pointer:** `int *p = NULL;` means "points to nothing". Always initialize pointers; test `if (p != NULL)` before use.
- **void pointer:** `void *v;` can hold the address of any type, but must be cast before dereferencing.

⚠ **Common Mistake**
Wild pointer: `int *p; *p = 5;` — p holds garbage, so the write goes to a random place (crash!).
Dangling pointer: pointing to memory that no longer exists (e.g. a local variable after its function returns).

Statement	x	p	*p	Explanation
`int x = 10;`	10	—	—	x created at address 1000
`int *p = &x;`	10	1000	10	p stores x's address
`*p = *p + 5;`	15	1000	15	value at 1000 changed → x changes
`x = 40;`	40	1000	40	x changed → *p sees 40

Why use pointers?	Benefit
Call by reference	A function can modify the caller's variables (Unit V)
Efficiency	Pass the address of big arrays / structures instead of copying
Data structures	Linked lists, trees, graphs are built from pointers



*Fig 4.2 — Pointer to pointer: int x = 10; int *p = &x; int **pp = &p;*

Pointer to pointer

```c
int x = 10, *p = &x, **pp = &p;
**pp = 99;                          // x changes through two levels
printf("%d %d %d", x, *p, **pp);
```

OUTPUT

```
99 99 99
```

4.2 Pointer and Address Arithmetic

Adding 1 to a pointer does **not** add 1 byte — it moves to the **next element**, i.e. adds `sizeof(type)` bytes:

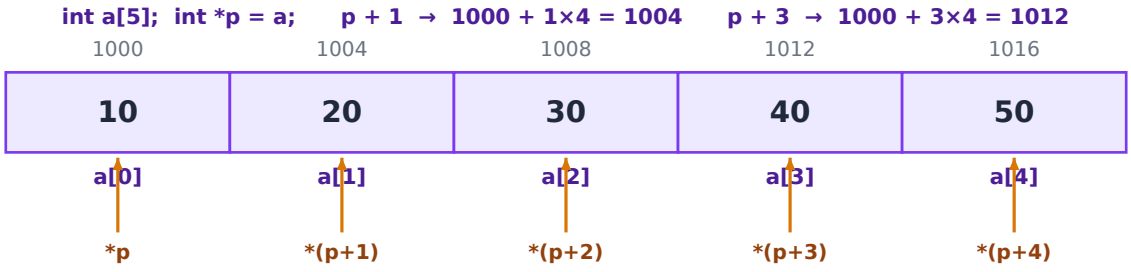


Fig 4.3 — Pointer arithmetic on an int array (each int = 4 bytes)

Operation	Allowed?	Meaning / Example
`p++`, `p--`, `++p`	Yes	Move to next / previous element
`p + n`, `p - n`	Yes	Address n elements ahead / behind: `*(p+2)` = third element
`p2 - p1`	Yes	Number of **elements** between two pointers (same array)
`p1 < p2`, `p1 == p2`	Yes	Compare positions
`p1 + p2`, `p * 2`, `p / 2`	**No**	Meaningless for addresses — compile error

★ Exam Focus
Scaling rule: new address = old address + n × sizeof(pointed type). `char *` moves 1 byte, `short *` 2, `int *` 4, `double *` 8. Precedence: `*p++` = value at p, then move p; `(*p)++` = increase the **value**; `*++p` = move first, then read.

Pointer expressions: *p++, (*p)++, *++p

```c
int a[] = {10, 20, 30}, *p = a;
printf("%d ", *p++);      // prints 10, THEN p moves to a[1]
(*p)++;                   // value at p increases: a[1] becomes 21
printf("%d ", *p);        // 21
printf("%d", *++p);       // p moves to a[2] FIRST, then prints 30
```

OUTPUT

```
10 21 30
```

4.3 Array Manipulation Using Pointers

The name of an array is the address of its first element, so these are all **equivalent**:

Element	Array notation	Pointer notation (p = a)	Address
1st	`a[0]`	`*a` or `*p`	`a` or `&a[0]`
i-th	`a[i]`	`*(a + i)` or `*(p + i)` or `p[i]`	`a + i` or `&a[i]`

Accessing with pointers

```c
int a[] = {10, 20, 30, 40, 50};
int *p = a;                              // no & needed: a already is an address
printf("%d %d %d\n", *p, *(p + 1), *(p + 4));
p++;                                     // p now points to a[1]
printf("%d\n", *p);
printf("%ld", (long)(&a[4] - &a[1]));    // pointer difference = elements
```

OUTPUT

```
10 20 50
20
3
```

Sum and maximum using pointers

```c
int a[5] = {3, 8, 1, 9, 4}, *p = a, *max = a, i, sum = 0;
for (i = 0; i < 5; i++) {
    sum += *(p + i);                     // same as a[i]
    if (*(p + i) > *max) max = p + i;    // remember ADDRESS of biggest
}
printf("Sum=%d Max=%d", sum, *max);
```

OUTPUT

```
Sum=25 Max=9
```

Reverse an array using two pointers

```c
int *l = a, *r = a + 4, t;               // l = first element, r = last element
while (l < r) {                          // pointer comparison
    t = *l;  *l = *r;  *r = t;           // swap the VALUES
    l++;  r--;
}
for (p = a; p < a + 5; p++) printf("%d ", *p);   // traverse
```

OUTPUT

```
4 9 1 8 3
```

String length using a pointer

```c
char *s = "Hello";  int len = 0;
while (*s != '\0') {                     // stop at the null character
    len++;
    s++;
}
printf("Length = %d", len);
```

OUTPUT

```
Length = 5
```

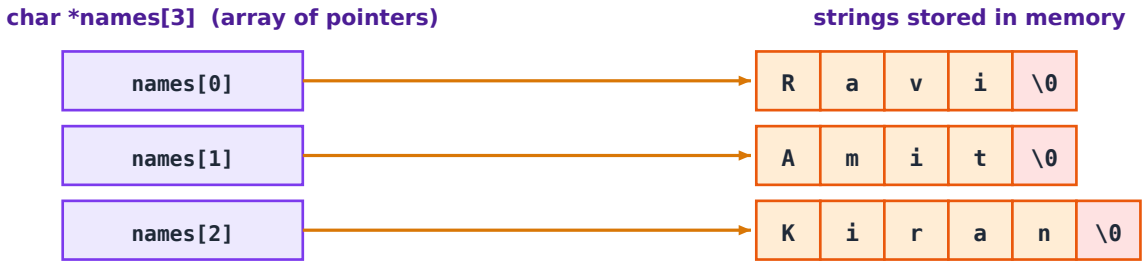


Fig 4.4 — Array of pointers: each element points to a string

String copy with pointers

```c
char src[] = "Hello", dst[10], *s = src, *t = dst;
while (*s != '\0') *t++ = *s++;     // copy char by char, move both pointers
*t = '\0';                          // terminate the new string
printf("%s", dst);
char *names[3] = {"Ravi", "Amit", "Kiran"};
printf(" %s", names[2]);
```

OUTPUT

```
Hello Kiran
```

	Array name `a`	Pointer `p`
Nature	Constant address of 1st element	Variable that stores an address
Can change?	`a++` or `a = p` → **error**	`p++`, `p = a` allowed
`sizeof`	whole array: `sizeof(a)` = 20 bytes (5 ints)	just the pointer: 8 bytes
Memory	Allocated automatically for all elements	Only 8 bytes; must point somewhere valid

● Real-life Use

Pointers allow **call by reference** (functions that change caller variables), fast array / string processing, and are the foundation of linked lists and trees.

4.4 User-Defined Data Types: Structures

◆ Definition

A **structure** (`struct`) groups **different types** of variables under one name. An array holds many values of the **same** type; a structure holds related values of **different** types (e.g. id, name, marks of a student).

Declaring and using a structure

```c
struct Student {            // 1. declare the TEMPLATE (no memory yet)
    int   id;
    char  name[20];
    float marks;
};                          // <-- semicolon is compulsory

struct Student s1 = {101, "Asha", 88.5};   // 2. create variable
s1.marks = 90.0;                           // 3. dot operator
printf("%d %s %.1f", s1.id, s1.name, s1.marks);
scanf("%d", &s1.id);                       // reading a member needs &
```

OUTPUT

```
101 Asha 90.0
```

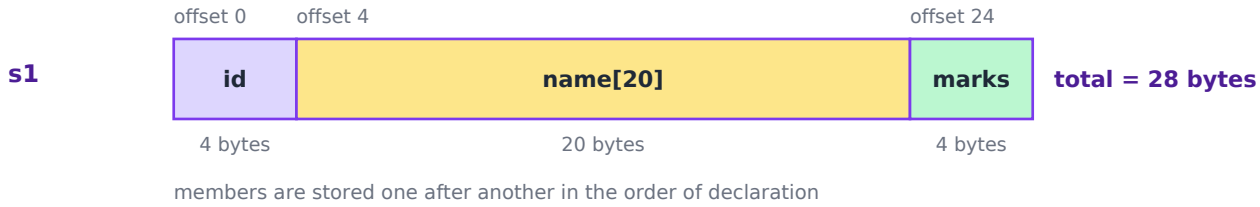


Fig 4.5 — Memory layout of struct Student s1

Style	Code	Note
Tagged	`struct Pt { int x, y; }; struct Pt a;`	Reusable template
With variables	`struct Pt { int x, y; } a, b;`	Declare template + variables
typedef	`typedef struct { int x, y; } Pt; Pt a;`	Short name, no struct keyword

Array	Structure
Same data type elements	Different data types allowed
Access by index `a[2]`	Access by member name `s.marks`
Elements are anonymous	Each member has its own name

Array of structures, nested structures, pointer to structure

Array of structures & arrow operator

```c
struct Student s[3] = {{1,"Ravi",72.5}, {2,"Amit",91.0},
                       {3,"Kiran",65.0}};
int i, top = 0;
for (i = 1; i < 3; i++)
    if (s[i].marks > s[top].marks) top = i;   // s[i].member
printf("Topper: %s (%.1f)\n", s[top].name, s[top].marks);

struct Student *ps = &s[0];       // POINTER to structure
printf("%s %.1f", ps->name, ps->marks);  // (*ps).x
```

OUTPUT

```
Topper: Amit (91.0)
Ravi 72.5
```

Nested structure and typedef

```c
struct Date { int d, m, y; };
struct Employee {
    char name[20];
    struct Date dob;                 // NESTED structure
};
struct Employee e = {"Meena", {14, 8, 2001}};
printf("%s born in %d", e.name, e.dob.y);        // chain of dots

typedef struct { int x, y; } Point;  // shorter name
Point p1 = {3, 4};                   // no 'struct' needed
```

OUTPUT

```
Meena born in 2001
```

- **Dot (.)** is used with a structure **variable**; **arrow (→)** with a structure **pointer**.
- Assigning `s2 = s1;` copies **all** members (even arrays inside). Structures cannot be compared with `==`.
- **Padding:** compilers align members, so `struct {char c; int i;}` occupies **8 bytes**, not 5. Always use `sizeof`.



Fig 4.6 — A pointer to a structure: members are reached with the arrow operator ->

Linear search using a pointer

```c
int a[] = {12, 45, 7, 30, 21}, *p, key = 30, found = 0;
for (p = a; p < a + 5; p++)        // p itself moves
    if (*p == key) { found = 1; break; }
if (found) printf("Found at position %ld", (long)(p - a) + 1);
else       printf("Not found");
```

OUTPUT

```
Found at position 4
```

4.5 User-Defined Data Types: Unions

◆ Definition

A **union** also groups members of different types, but **all members share the same memory**. Its size is that of its **largest member**, and only **one member** holds a valid value at a time.

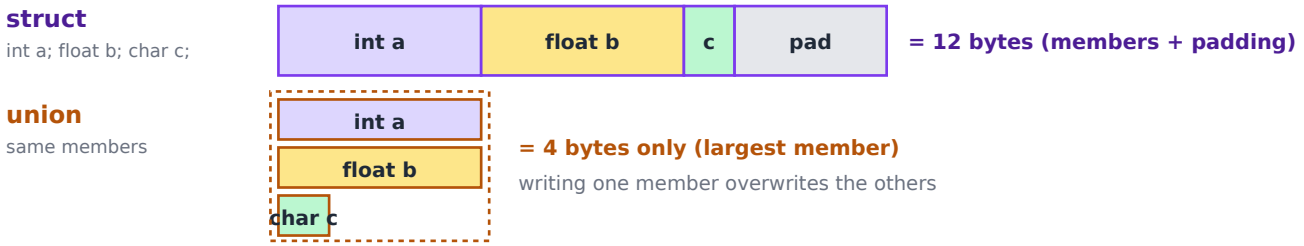


Fig 4.7 — Memory of struct vs union with the same members

Using a union

```c
union Data { int i; float f; char c; };     // declared like a struct
union Data u;
u.i = 65;
printf("%d %c\n", u.i, u.c);                // 65 A  (same byte read as char)
u.f = 3.5;                                   // overwrites u.i
printf("%.1f\n", u.f);
printf("size = %zu", sizeof(u));             // largest member = 4
```

OUTPUT

```
65 A
3.5
size = 4
```

Feature	Structure	Union
Keyword	`struct`	`union`
Memory	Each member has its **own** space	All members **share** one space
Size	Sum of all members (+ padding)	Size of the **largest** member
Active members	All members valid together	Only the **last written** member is valid
Use when	Records: student, employee, point	Saving memory; variant data (packet, register)

● Real-life Use

Structures: student / employee records, linked-list nodes, coordinates, bank accounts. **Unions:** embedded hardware registers, network packet formats, interpreters that store "an int *or* a float *or* a char".

Tagged union (struct + union together)

```c
struct Shape {
    int type;               // TAG: 1 = int, 2 = float
    union { int i; float f; } val;
};
struct Shape v = {2, .val.f = 3.14f};
if (v.type == 1) printf("%d", v.val.i);
else             printf("%.2f", v.val.f);
```

OUTPUT

```
3.14
```

Predict the output (practice)

#	Code	Output
1	`int x=5, *p=&x; *p = *p * 2; printf("%d", x);`	10
2	`int a[]={1,2,3}, *p=a+1; printf("%d", *p + *(p+1));`	5 (2 + 3)
3	`int a[]={4,5,6}; printf("%d", *(a+2) - *a);`	2 (6 − 4)
4	`struct P {int x,y;} p={3,4}; printf("%d", p.x*p.y);`	12
5	`union U {int i; char c;} u; u.i=66; printf("%c", u.c);`	B (ASCII 66)
6	`printf("%zu", sizeof(int *));`	8 on 64-bit systems

⚠ Common Mistake
(1) Forgetting the **;** after the closing brace of a struct/union; (2) using `.` instead of `->` on a pointer; (3) reading a union member other than the one last written; (4) dereferencing a NULL or uninitialized pointer; (5) `p + 1` on `int *` moves 4 bytes, not 1.

struct	Members	Raw sum	sizeof (typical)	Reason
`{char c; int i;}`	char, int	5	8	3 bytes padding after c
`{int i; char c;}`	int, char	5	8	padding at the end
`{char a; char b; int i;}`	char, char, int	6	8	order members large → small to save space

4.6 More Solved Programs on Pointers, Structures and Unions

Count even and odd numbers using a pointer

```c
int a[6] = {4, 7, 10, 13, 16, 19}, *p = a, even = 0, odd = 0;
while (p < a + 6) {                    // p walks through the array
    if (*p % 2 == 0) even++; else odd++;
    p++;
}
printf("Even=%d Odd=%d", even, odd);
```

OUTPUT

```
Even=3 Odd=3
```

Swapping two variables using pointers

```c
int a = 10, b = 20, *p = &a, *q = &b, t;
t = *p;  *p = *q;  *q = t;              // swap the VALUES through pointers
printf("a=%d b=%d", a, b);
```

OUTPUT

```
a=20 b=10
```

Average marks using an array of structures

```c
struct Student { char name[10]; float marks; };
struct Student s[3] = {{"Ravi", 72.5}, {"Amit", 91.0}, {"Kiran", 65.0}};
float total = 0;  int i;
for (i = 0; i < 3; i++) total += s[i].marks;
printf("Average = %.2f", total / 3);
```

OUTPUT

```
Average = 76.17
```

Structure program: adding complex numbers

```c
struct Complex { float re, im; };
struct Complex a = {3, 4}, b = {1, 2}, c;
c.re = a.re + b.re;   c.im = a.im + b.im;          // (3+4i) + (1+2i)
printf("Sum = %.0f + %.0fi", c.re, c.im);
```

OUTPUT

```
Sum = 4 + 6i
```

Costliest book (array of structures + pointer)

```c
struct Book { char title[15]; float price; };
struct Book b[3] = {{"C Basics", 350}, {"Maths", 520}, {"Physics", 450}};
struct Book *costly = &b[0];  int i;
for (i = 1; i < 3; i++)
    if (b[i].price > costly->price) costly = &b[i];  // best so far
printf("%s costs %.0f", costly->title, costly->price);
```

OUTPUT

```
Maths costs 520
```

Only one union member is valid at a time

```c
union Num { int i; float f; };
union Num n;
n.i = 7;   printf("%d\n", n.i);          // int is valid now
n.f = 2.5; printf("%.1f", n.f);          // float is valid now (int is lost)
```

OUTPUT

```
7
2.5
```

Tricky question	Answer
`int *p, q;`	p is a pointer to int; **q is a plain int** (the * binds to p only)
`int *p = 5;`	Wrong: a pointer must hold an address, not a number
Can a pointer point to a pointer?	Yes: `int **pp = &p;`
Is `p[i]` valid for a pointer?	Yes, it is just `*(p + i)`

★ UNIT SUMMARY — QUICK REVISION

- A **pointer** stores an **address**. `&x` = address of x; `*p` = value at the address in p. Initialize with an address or **NULL**.
- **Pointer arithmetic** is scaled: `p + n` = address + n×sizeof(type). Allowed: ++, --, +n, −n, p2−p1, comparison.
- `a[i]` ≡ `*(a + i)` ≡ `*(p + i)`. Array name is a constant address; a pointer variable can move.
- **Structure**: different types, own memory each, access with **.**; through a pointer use **->**. Can be nested, arrays of struct, and renamed with `typedef`.
- **Union**: shared memory, size = largest member, only one valid member at a time.
- Pointer size = 8 bytes (64-bit) regardless of type; `sizeof(struct)` may include padding.

Q Exam-Oriented Questions

Short Answer (2 marks)

1. What is a pointer? How is it declared?
2. What are the address-of (&) and dereference (*) operators?
3. What is a NULL pointer? Why is it useful?
4. Explain wild and dangling pointers.
5. If `int *p` holds 2000, what is `p + 3`? Why?
6. Write the equivalent pointer expression for `a[i]`.
7. Differentiate between `.` and `->` operators.
8. What is padding in a structure?
9. What is typedef? Give an example.
10. Why does a union need less memory than a structure?

Long Answer / Programs (5–10 marks)

1. Explain pointers with a diagram, covering declaration, initialization and dereferencing.
2. Explain pointer arithmetic with examples. Which operations are not allowed?
3. Write programs using pointers to (a) find the sum and largest of an array, (b) reverse an array, (c) find string length.
4. Define a structure. Write a program to read and print details of n students and find the topper.
5. Explain nested structures and pointer to structure with a program.
6. Explain unions with an example. Compare structure and union with a memory diagram.
7. Explain the relationship between arrays and pointers. How does an array name differ from a pointer?

8. Write a program using an array of structures to store 5 books (title, price) and display the costliest book.

Σ Quick Reference: Syntax Cheat-Sheet

Concept	Syntax	Example
Declare pointer	`type *name;`	`int *p;`
Address of	`&variable`	`p = &x;`
Dereference	`*pointer`	`*p = 25; y = *p;`
Pointer arithmetic	new address = old + n × sizeof(type)	`p + 2` (int: +8 bytes)
Array ↔ pointer	`a[i] = *(a + i)`	`p = a; *(p + 3)`
Define structure	`struct Tag { members; };`	`struct Pt { int x, y; };`
Access member	`var.member` / `ptr->member`	`s.id` / `ps->id`
Define union	`union Tag { members; };`	`union U { int i; float f; };`
Rename a type	`typedef old new;`	`typedef struct {int x,y;} Pt;`

INTRODUCTION TO PROGRAMMING • DIGITAL NOTES

UNIT V

Functions & File Handling

V

Functions • Call by value / reference • Arrays as parameters • Scope & lifetime • File handling

▶ What you will learn in this unit

- Introduction to functions
- Declaration, definition & call
- Return types & arguments
- Call by value vs call by reference
- Modifying parameters with pointers
- Arrays as parameters
- Parameters & return values
- Scope of variables
- Lifetime & storage classes
- Basics of file handling
- fopen modes & file functions
- Exam questions

5.1 Introduction to Functions

◆ Definition

A **function** is a named, self-contained block of code that performs one specific task and can be **called** whenever needed. Every C program has at least one function: `main()`.

Benefit	Meaning
Modularity	Big problem → small functions (top-down design from Unit I)
Reusability	Write once, call many times
Easy debugging	Test and fix each function separately
Readability	Short main() that reads like a list of steps

Library functions	User-defined functions
Already written, declared in header files: `printf, scanf, sqrt, strlen, toupper`	Written by the programmer for a specific need: `add(), isPrime(), grade()`
Just `#include` the header and call	Must be declared, defined and called

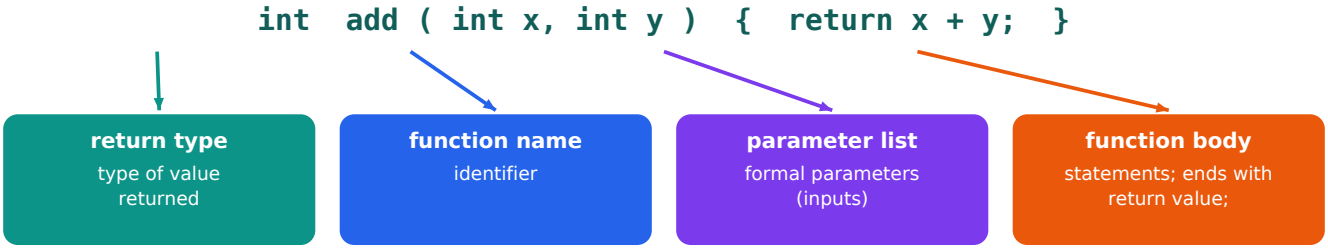


Fig 5.1 — Anatomy of a function definition

5.2 Function Declaration, Definition and Call

Step	What it is	Example
Declaration (prototype)	Tells the compiler the name, return type and parameter types **before** use; ends with ;	`int add(int, int);`
Definition	The actual code of the function	`int add(int x, int y) { return x + y; }`
Call	Transfers control to the function with **actual arguments**	`r = add(5, 3);`

Complete program with a user-defined function

```c
#include <stdio.h>
int add(int x, int y);            // 1. DECLARATION (prototype)

int main() {
    int r = add(5, 3);            // 3. CALL: 5 and 3 are ACTUAL arguments
    printf("Sum = %d", r);
    return 0;
}
int add(int x, int y) {           // 2. DEFINITION: x, y are FORMAL parameters
    return x + y;                 // sends the result back to the caller
}
```

OUTPUT

```
Sum = 8
```

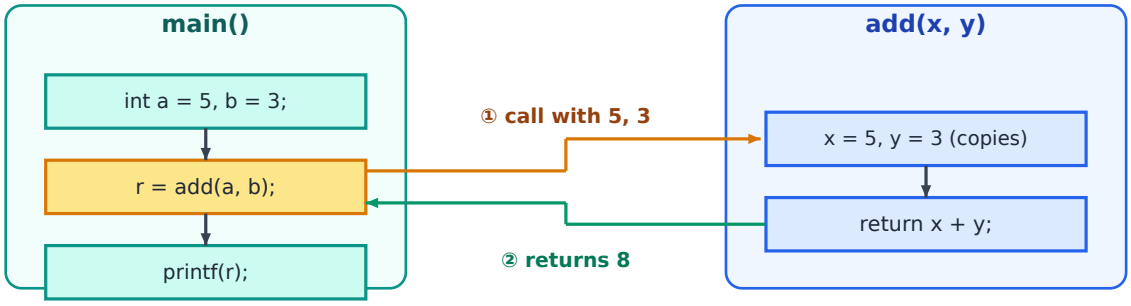


Fig 5.2 — Control flow of a function call: main → add → back to main

Return types and arguments — four kinds of functions

Type	Arguments	Return	Example
1	No	No	`void greet(void) { printf("Hello"); }`
2	**Yes**	No	`void square(int n) { printf("%d", n*n); }`
3	No	**Yes**	`int readNum(void) { int n; scanf("%d",&n); return n; }`
4	**Yes**	**Yes**	`int cube(int n) { return n*n*n; }` ← most useful

- **Return type**: `int, float, char, ...` or `void` (returns nothing). A function returns **one** value (use pointers for more).
- **Formal parameters** are in the definition; **actual arguments** are in the call. They match in number, order and type.
- **return** immediately ends the function. If the return type is not void, every path must return a value.
- If a function is defined **before** main(), the separate prototype is optional — but good practice.

A useful function: isPrime()

```c
int isPrime(int n) {
    int i;
    if (n < 2) return 0;                 // early return = "not prime"
    for (i = 2; i * i <= n; i++)
        if (n % i == 0) return 0;
    return 1;                            // reached only if no divisor found
}
// in main:  printf("%d %d", isPrime(7), isPrime(10));
```

OUTPUT

```
1 0
```

5.3 Parameter Passing: Value vs Reference (Using Pointers)

Call by value: the function gets a **copy** of each argument — changes stay inside the function. **Call by reference:** the function gets the **address** (pointer) — it can change the caller's variables.

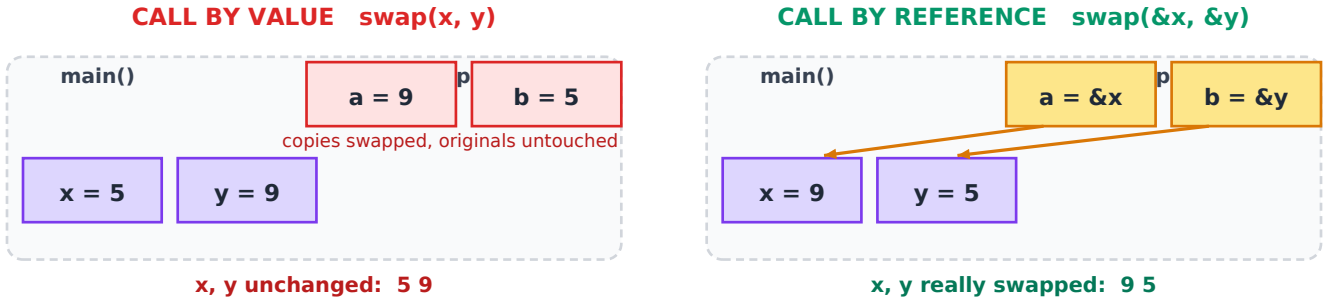


Fig 5.3 — Why swapping fails with values but works with pointers

Swap: by value and by reference

```c
void swapV(int a, int b)   { int t = a;  a = b;    b = t; }     // copies
void swapR(int *a, int *b) { int t = *a; *a = *b;  *b = t; }    // addresses

int main() {
    int x = 5, y = 9;
    swapV(x, y);    printf("%d %d\n", x, y);    // no effect
    swapR(&x, &y);  printf("%d %d\n", x, y);    // & passes the address
    return 0;
}
```

OUTPUT

```
5 9
9 5
```

	Call by value	Call by reference (pointers)
Passed	Copy of the value	Address of the variable
Formal parameter	Ordinary variable `int a`	Pointer `int *a`
Call	`f(x)`	`f(&x)`
Changes affect caller?	**No**	**Yes** (via `*a`)
Use for	Read-only inputs	Modify caller's data, return several results

Returning multiple values through pointers

```c
void minmax(int a[], int n, int *min, int *max) {   // returns TWO results
    int i;
    *min = *max = a[0];
    for (i = 1; i < n; i++) {
        if (a[i] < *min) *min = a[i];
        if (a[i] > *max) *max = a[i];
    }
}
// main:  int a[] = {42,7,56,19,33}, lo, hi;
//        minmax(a, 5, &lo, &hi);   printf("Min=%d Max=%d", lo, hi);
```

OUTPUT

```
Min=7 Max=56
```

5.4 Arrays as Function Parameters

- Only the **address of the first element** is passed (no copy!), so the function works on the **original array** — it can modify it.
- The array **size is not passed** automatically — send it as a second argument `n`.
- `int a[]` and `int *a` mean exactly the same in a parameter list.
- For 2-D arrays the **column size must be given**: `void show(int m[][3], int rows)`.

Passing a 1-D array

```c
int sum(int a[], int n) {                // a[] == int *a
    int i, s = 0;
    for (i = 0; i < n; i++) s += a[i];
    return s;
}
void doubleAll(int a[], int n) {         // modifies the ORIGINAL array
    int i;
    for (i = 0; i < n; i++) a[i] *= 2;
}
// main:  int arr[] = {1,2,3,4,5};
//        printf("%d ", sum(arr, 5));       // call: array NAME, no []
//        doubleAll(arr, 5);
//        printf("%d", sum(arr, 5));
```

OUTPUT

```
15 30
```

Passing 2-D arrays and strings

```c
void show(int m[][3], int rows) {        // columns (3) must be specified
    int i, j;
    for (i = 0; i < rows; i++) {
        for (j = 0; j < 3; j++) printf("%d ", m[i][j]);
        printf("\n");
    }
}
int length(char s[]) {                   // strings too
    int n = 0;  while (s[n] != '\0') n++;  return n;
}
// show(matrix, 2);      length("Hello") -> 5
```

OUTPUT

```
1 2 3
4 5 6
```

5.5 Scope and Lifetime of Variables

Term	Meaning	Question it answers
Scope	The region of the program where a variable can be **accessed**	WHERE can I use it?
Lifetime	The period during **run time** for which the variable exists in memory	WHEN does it exist?

Kind	Declared	Scope	Lifetime	Default value
Local	Inside a function / block	Only that function / block	Until the block ends	Garbage
Global	Outside all functions	Whole file (after declaration)	Entire program run	0
Formal parameter	In function header	That function	During the call	Value of argument
static local	Local with `static`	That function only	**Entire program run**	0

Scope, shadowing and static

```c
int g = 10;                          // GLOBAL: visible everywhere below

void f() {
    int g = 20;                      // LOCAL g hides (shadows) global g
    printf("%d\n", g);
}
void counter() {
    int a = 0;  a++;                 // auto: recreated each call
    static int s = 0;  s++;          // static: created ONCE, remembers
    printf("a=%d s=%d\n", a, s);
}
// main: f(); printf("%d\n", g); counter(); counter(); counter();
```

OUTPUT

```
20
10
a=1 s=1
a=1 s=2
a=1 s=3
```

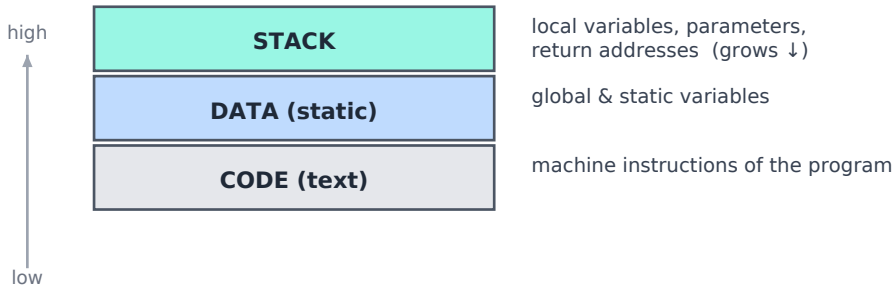


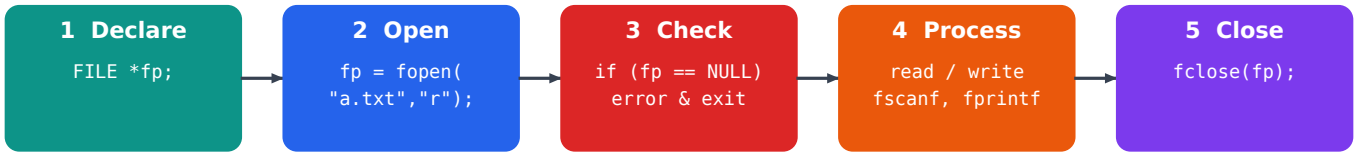
Fig 5.4 — Where variables live in memory (program layout)

Storage class	Keyword	Stored in	Default	Scope / Lifetime	Use
Automatic	`auto`	Stack	garbage	block / block	Normal local variables
Register	`register`	CPU register (hint)	garbage	block / block	Fast loop counters; cannot use &
Static	`static`	Data segment	0	block or file / **whole run**	Keep value between calls
External	`extern`	Data segment	0	global / whole run	Share a variable across files

5.6 Basics of File Handling

◆ Definition

A **file** stores data **permanently** on disk (variables vanish when the program ends). File handling lets a program **create, read, write and append** files using a **file pointer** of type `FILE *`.



Always follow these five steps — an unclosed file may lose its buffered data

Fig 5.5 — Steps in file processing



Fig 5.6 — Data flows through a buffer between program and disk

Mode	Meaning	If file does not exist	If file exists
`"r"`	Read only	**fopen returns NULL**	Opens at beginning
`"w"`	Write only	Creates new file	**Erases** old contents
`"a"`	Append (write at end)	Creates new file	Keeps data, adds at end
`"r+"` / `"w+"` / `"a+"`	Read and write	r+: NULL; w+, a+: create	w+ erases; a+ appends

Function	Purpose	Function	Purpose
`fopen(name, mode)`	Open file, returns FILE *	`fclose(fp)`	Close file
`fprintf(fp, fmt, ...)`	Formatted write	`fscanf(fp, fmt, ...)`	Formatted read
`fputc(c, fp)`	Write one char	`fgetc(fp)`	Read one char (EOF at end)
`fputs(s, fp)`	Write a string	`fgets(s, n, fp)`	Read a line
`feof(fp)`	True at end of file	`perror(msg)`	Prints reason for a failed open

Writing to a text file

```c
FILE *fp = fopen("data.txt", "w");          // "w": create / overwrite
if (fp == NULL) { printf("Cannot open file"); return 1; }
fprintf(fp, "Asha %d\n", 85);               // like printf, but into the file
fputs("Ravi 72\n", fp);
fclose(fp);                                 // data.txt now has 2 lines
```

OUTPUT

```
data.txt:
Asha 85
Ravi 72
```

Reading a text file

```c
char name[20];  int m;
fp = fopen("data.txt", "r");
while (fscanf(fp, "%s %d", name, &m) == 2)   // 2 items read?
    printf("%s scored %d\n", name, m);
fclose(fp);
```

OUTPUT

```
Asha scored 85
Ravi scored 72
```

Character I/O, counting lines, appending

```c
int ch, lines = 0;
fp = fopen("data.txt", "r");
while ((ch = fgetc(fp)) != EOF) {          // until end of file
    putchar(ch);
    if (ch == '\n') lines++;
}
fclose(fp);
fp = fopen("data.txt", "a");               // APPEND mode
fputs("Kiran 65\n", fp);  fclose(fp);
```

OUTPUT

```
Asha 85
Ravi 72
lines = 2
```

Write n numbers to a file, read them back, find the sum

```c
int n, i, x, sum = 0;
FILE *fp = fopen("nums.txt", "w");
printf("How many numbers? ");  scanf("%d", &n);
for (i = 0; i < n; i++) { scanf("%d", &x); fprintf(fp, "%d\n", x); }
fclose(fp);                                // numbers saved on disk
fp = fopen("nums.txt", "r");
while (fscanf(fp, "%d", &x) == 1) sum += x;   // read back
fclose(fp);
printf("Sum = %d", sum);
```

OUTPUT

```
How many numbers? 3
10 20 30
Sum = 60
```

★ Exam Focus

Remember: (1) always check `fp == NULL`; (2) mode `"w"` destroys old data; (3) `fclose()` flushes the buffer; (4) EOF is an **int** constant — store `fgetc()` result in `int`, not `char`; (5) `stdin, stdout, stderr` are automatically opened file pointers for keyboard, screen and error output.

More function examples

```c
void grade(int m) {                        // type 2: arguments, no return
    if (m >= 75)      printf("Distinction\n");
    else if (m >= 40) printf("Pass\n");
    else              printf("Fail\n");
}
long power(int b, int e) {                 // type 4: arguments + return
    long r = 1;  while (e-- > 0) r *= b;  return r;
}
// main: grade(82); grade(35); printf("%ld", power(2, 10));
```

OUTPUT

```
Distinction
Fail
1024
```

Copy a file and count characters / words

```c
FILE *in = fopen("data.txt", "r");
FILE *out = fopen("copy.txt", "w");
int ch, chars = 0, words = 0, inword = 0;
if (in == NULL || out == NULL) { perror("fopen"); return 1; }
while ((ch = fgetc(in)) != EOF) {
    fputc(ch, out);                                  // copy the file
    chars++;
    if (ch == ' ' || ch == '\n') inword = 0;
    else if (!inword) { inword = 1; words++; }   // new word
}
fclose(in);  fclose(out);
printf("chars=%d words=%d", chars, words);
```

OUTPUT

```
chars=16 words=4
```

Predict the output (practice)

#	Code	Output
1	`int f(int x){ return x*2; } printf("%d", f(f(3)));`	12
2	`void g(int a){ a = 99; } int x=5; g(x); printf("%d", x);`	5 (call by value)
3	`void h(int *a){ *a = 99; } int x=5; h(&x); printf("%d", x);`	99 (call by reference)
4	`void k(){ static int c=0; c++; printf("%d", c); } k(); k();`	12
5	`int a[3]={1,2,3}; void f(int x[]){ x[0]=9; } f(a); printf("%d", a[0]);`	9 (array passed by address)
6	`fopen("nofile.txt", "r")` when the file does not exist	returns NULL

Common mistake	Fix
Calling a function before declaring it	Add a prototype above main() or define it earlier
Forgetting `&` when the function expects a pointer	Call `swap(&x, &y)`, not `swap(x, y)`
Returning the address of a local variable	Local memory dies on return → dangling pointer; use a static variable
Not closing a file / not checking NULL	Always test `fp == NULL` and call `fclose(fp)`
Same name for a local and a global variable	The local hides the global (shadowing) — use distinct names

Stream	File pointer	Connected to	Example
Standard input	`stdin`	Keyboard	`fgets(s, 50, stdin);`
Standard output	`stdout`	Screen	`fprintf(stdout, "Hi");`
Standard error	`stderr`	Screen (error channel)	`fprintf(stderr, "Error!");`

● Real-life Use

Saving high scores and settings, student / bank records, log files, reading CSV data, configuration files, and exchanging data between programs.

★ UNIT SUMMARY — QUICK REVISION

- **Function** = reusable block: **declaration** (prototype), **definition**, **call**. Four kinds by arguments/return.
- **Call by value** copies; **call by reference** passes addresses (`f(&x)`, `int *p`) so the function can change the caller's variables.
- **Arrays** are passed as addresses (original is modified); pass the size separately; 2-D arrays need the column size.
- **Scope** = where visible (local / global); **lifetime** = how long it exists. `static` keeps its value between calls. Storage classes: auto, register, static, extern.
- **Files:** `FILE *fp` → `fopen` → check NULL → `fprintf/fscanf/fgetc/fputc/fgets/fputs` → `fclose`. Modes: r, w, a (+ b for binary).

Q Exam-Oriented Questions

Short Answer (2 marks)

1. What is a function? Mention two advantages.
2. Differentiate between function declaration and definition.
3. Differentiate between formal and actual parameters.
4. What is a function prototype? Why is it needed?
5. Differentiate between call by value and call by reference.
6. How are arrays passed to a function?
7. What is the difference between scope and lifetime?
8. What is a static variable? Give a use.
9. List the file opening modes in C.
10. What happens if `fopen()` fails to open a file?

Long Answer / Programs (5–10 marks)

1. Explain the types of user-defined functions based on arguments and return values, with examples.
2. Write a program to swap two numbers using call by value and call by reference. Explain the output.
3. Write a function to find the largest element of an array and call it from main().
4. Write functions to check even/odd and to compute a factorial using a loop; call them from main().
5. Explain local, global, static, auto, register and extern with examples.
6. Write a program to write n numbers to a file and then read them back and display the sum.
7. Explain file operations (open, read, write, append, close) with syntax and examples.
8. Write a program to count characters, words and lines in a file.

Σ Quick Reference: Syntax Cheat-Sheet

Concept	Syntax
Prototype	`return_type name(type1, type2);`
Definition	`return_type name(type1 p1, type2 p2) { ...; return value; }`
Call	`result = name(arg1, arg2);`
Call by reference	definition: `void f(int *p)` call: `f(&x);` use: `*p = 10;`
Array parameter	`int f(int a[], int n)` or `int f(int *a, int n)` call: `f(arr, 5)`
2-D array parameter	`void f(int m[][COLS], int rows)`
Static local	`static int count = 0;` (initialized once, keeps value)
Open / close file	`FILE *fp = fopen("f.txt", "r"); if (fp == NULL) ...; fclose(fp);`
Read / write text	`fscanf(fp, "%d", &n); fprintf(fp, "%d\n", n);`
Read until end	`while ((ch = fgetc(fp)) != EOF) { ... }`

Function checklist	Scope checklist
1. Prototype before main? 2. Argument count/types match? 3. Return type matches returned value?	Local: inside block, dies at } Global: whole file, lives till program ends Static: local scope, global lifetime